



GRINDOS FLAGSHIP SERIES

System Design Handbook

From Fundamentals to Interview Readiness

Created by Pranav Gawai

System Design Foundations

This handbook serves as a comprehensive guide to system design, transitioning from core fundamentals to interview readiness. We organize the content into 10 structured sections covering networking, databases, messaging, microservices, storage, security, observability, case studies, and interview frameworks.

System Architecture & Scalability

Understanding tradeoffs between consistency and availability (CAP theorem), horizontal and vertical scaling, and caching architectures.

Microservice Orchestrations

Tracing requests across distributed services, applying circuit breakers, service meshes, rate limiting, and event-driven queues.

Index: Foundations to Messaging

Section 1: Foundations

- 01. What Is System Design Page 05
- 02. Scalability Page 06
- 03. Availability Page 07
- 04. Reliability Page 08
- 05. Fault Tolerance Page 09
- 06. Latency Page 10
- 07. Throughput Page 11
- 08. CAP Theorem Page 12
- 09. Consistency Models Page 13
- 10. Horizontal vs Vertical Page 14

Section 2: Networking

- 11. Load Balancer Page 15
- 12. Reverse Proxy Page 16
- 13. CDN Page 17
- 14. API Gateway Page 18
- 15. Service Discovery Page 19
- 16. DNS Flow Page 20
- 17. Caching Basics Page 21

Section 3: Databases

- 18. SQL vs NoSQL Page 22
- 19. Database Indexing Page 23
- 20. Replication Page 24
- 21. Read Replicas Page 25
- 22. Sharding Page 26
- 23. Partitioning Page 27
- 24. CQRS Page 28
- 25. Eventual Consistency Page 29
- 26. Distributed Transactions Page 30

Section 4: Messaging

- 27. Message Queues Page 31
- 28. Kafka Page 32
- 29. RabbitMQ Page 33
- 30. Pub/Sub Page 34
- 31. Event Driven Architecture Page 35
- 32. Dead Letter Queue Page 36
- 33. Retry Patterns Page 37
- 34. Idempotency Page 38

Index: Microservices to Interview Framework

Section 5: Microservices

- 35. Monolith vs Microservices ... Page 39
- 36. Circuit Breaker Page 40
- 37. Service Mesh Page 41
- 38. Saga Pattern Page 42
- 39. Distributed Tracing Page 43
- 40. Rate Limiting Page 44

Section 6: Storage

- 41. Object Storage Page 45
- 42. Blob Storage Page 46
- 43. File Upload Architecture Page 47
- 44. Media Processing Page 48
- 45. Search Architecture Page 49

Section 7: Security

- 46. Authentication Page 50
- 47. Authorization Page 51
- 48. JWT Page 52
- 49. OAuth Page 53
- 50. Session Auth Page 54
- 51. RBAC Page 55
- 52. ABAC Page 56
- 53. Zero Trust Page 57

Section 8: Observability

- 54. Logging Page 58
- 55. Monitoring Page 59
- 56. Metrics Page 60
- 57. Alerting Page 61
- 58. Tracing Page 62
- 59. Prometheus Page 63
- 60. Grafana Page 64

Section 9: Case Studies

- 61. URL Shortener Page 65
- 62. WhatsApp Page 66
- 63. Instagram Feed Page 67
- 64. YouTube Page 68
- 65. Uber Page 69
- 66. Zomato Page 70
- 67. Google Drive Page 71
- 68. Notification System Page 72
- 69. Chat System Page 73
- 70. Rate Limiter Page 74

Section 10: Framework

- 71. Requirement Gathering Page 75
- 72. Capacity Estimation Page 76
- 73. API Design Page 77
- 74. Database Design Page 78
- 75. Bottleneck Analysis Page 79
- 76. Scaling Strategy Page 80
- 77. Tradeoff Discussion Page 81
- 78. Common Mistakes Page 82
- 79. Interview Framework Page 83
- 80. Top 100 Questions Page 84

What Is System Design

📖 DEFINITION & WHY IT EXISTS

Definition: The process of defining the architecture, interfaces, and data models for a system to satisfy specific requirements.

Why It Exists: It enables building scalable, reliable, and cost-effective software systems that can handle growth over time.

🌐 REAL WORLD USAGE & TRADEOFF

Usage: Used by companies to transition from simple MVPs to distributed systems serving millions of users.

Trade-off: Upfront architecture planning vs rapid product release velocity.

⚠️ COMMON MISTAKE

Neglecting non-functional requirements like latency and availability during early planning.

🎯 INTERVIEWER TRAP

What is the difference between functional and non-functional requirements?

💡 TOP FOLLOW-UP

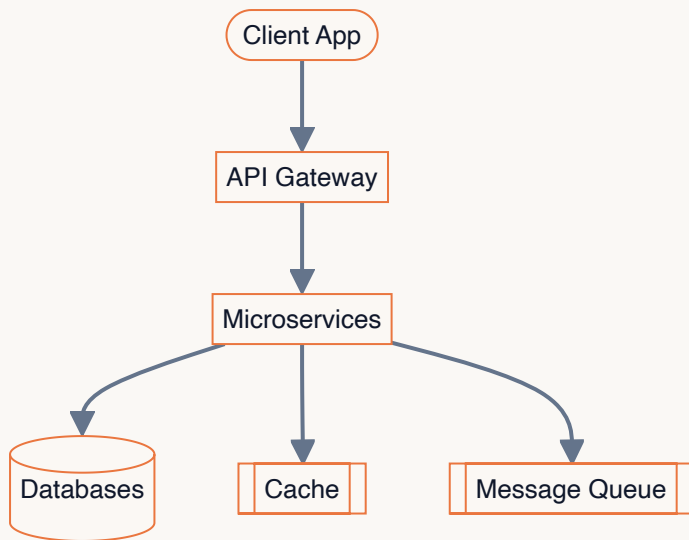
How do you handle capacity estimation for storage?

💡 MEMORY TRICK

System design is like town planning: define roads (APIs) and zones (services) before building houses.

System Design Architecture

ARCHITECTURE



Eraser Diagram Spec

```
// 1. Define nodes and logical groups
Client App [icon: smartphone, color: orange]
API Gateway [icon: server, color: blue]
Microservices [icon: code, color: blue]
Databases [icon: database, color: green]
Cache [icon: layers, color: yellow]
Message Queue [icon: list, color: yellow]
```

```
// 2. Define connections
Client App > API Gateway
API Gateway > Microservices
Microservices > Databases
Microservices > Cache
Microservices > Message Queue
```

Component Breakdown

Client App The frontend interface.

API Gateway Central entry point handling routing.

Microservices Independent business logic servers.

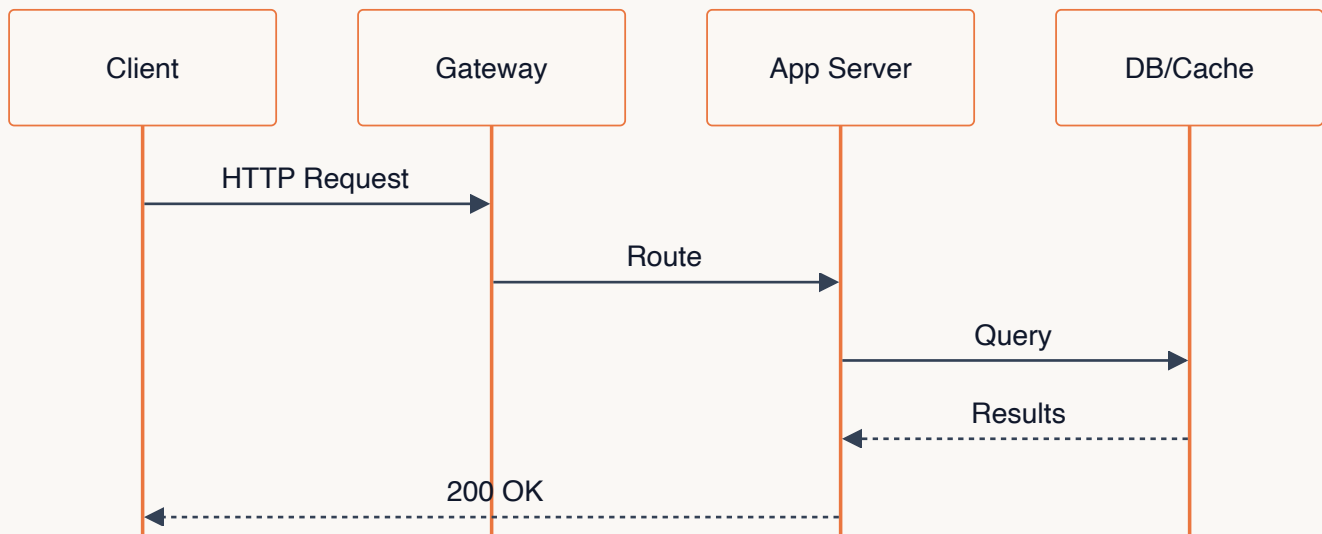
Databases Persistent storage.

Visual Layout Description

The client sits at the top, directing traffic through a central gateway which fans out to internal services and storage layers.

System Design Request Flow

REQUEST FLOW



◇ Eraser Diagram Spec

// Sequence

Client > Gateway: HTTP Request

Gateway > App Server: Route

App Server > DB: Query

DB > App Server: Results

App Server > Client: 200 OK

■ Component Breakdown

HTTP Request Standard REST or gRPC call.

Route Gateway forwards packet based on path.

Query Service fetches required state.

▣ Visual Layout Description

Vertical sequence showing round-trip synchronous request processing.

Request Flow: Client → Gateway → Application Server → DB / Cache lookup → Client Response.

Pros: Ensures scalability, cost management, and maintainability.

Cons: Adds design complexity and upfront development time overhead.

Scalability

DEFINITION & WHY IT EXISTS

Definition: A system's ability to handle growing amounts of work by adding resources to the system.

Why It Exists: Prevents system lockups during peak user traffic spikes.

REAL WORLD USAGE & TRADEOFF

Usage: E-commerce platforms scaling compute instances during holiday shopping sales.

Trade-off: Over-provisioning servers (high cost) vs active auto-scaling lag (brief downtime).

COMMON MISTAKE

Assuming scaling is instant. Container spin-up times can take several minutes.

INTERVIEWER TRAP

How do you measure scalability?

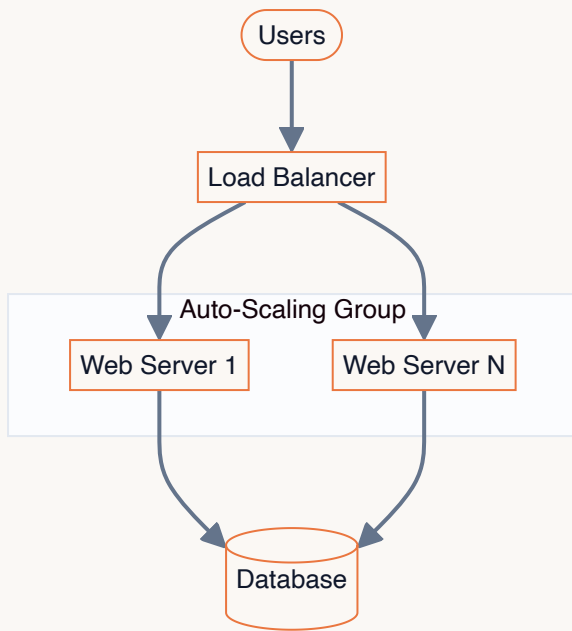
TOP FOLLOW-UP

What are the limits of vertical scalability?

MEMORY TRICK

Scalability is adding more checkout lanes at a supermarket when lines get long.

Horizontal Scaling Architecture ARCHITECTURE



Eraser Diagram Spec

```
// Nodes
Users [icon: users, color: orange]
Load Balancer [icon: server, color: blue]
Group Auto-Scaling Group {
  Web Server 1 [icon: box, color: blue]
  Web Server N [icon: box, color: blue]
}
Database [icon: database, color: green]
```

```
// Edges
Users > Load Balancer
Load Balancer > Web Server 1
Load Balancer > Web Server N
Web Server 1 > Database
Web Server N > Database
```

Component Breakdown

Auto-Scaling Group Dynamically adds/removes instances.

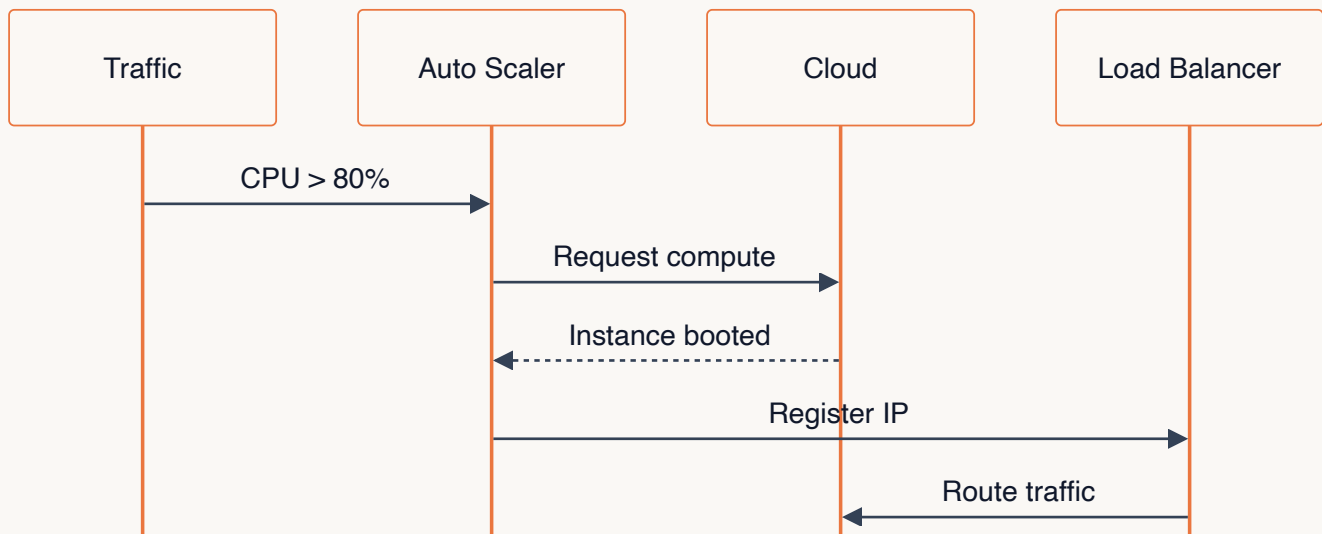
Load Balancer Distributes traffic across active instances.

Visual Layout Description

Traffic flows from users, splits at the load balancer, and converges at the database.

Auto-Scaling Flow

REQUEST FLOW



Eraser Diagram Spec

Traffic > Auto Scaler: CPU > 80%

Auto Scaler > Cloud: Request compute

Cloud > Auto Scaler: Instance booted

Auto Scaler > Load Balancer: Register IP

Load Balancer > Cloud: Route traffic

Component Breakdown

TriggerMetric threshold exceeded.

ProvisionCloud allocates resources.

RegisterLB adds instance to pool.

Visual Layout Description

Shows the background asynchronous process of scaling infrastructure.

Request Flow: Traffic Spikes -> Auto-scaler triggers -> New instances join load balancer pool -> Load balances.

Pros: Prevents downtime, improves load speeds, and supports user growth.

Cons: Increases cloud infrastructure costs and data synchronization lag.

Availability

DEFINITION & WHY IT EXISTS

Definition: The percentage of time a system remains operational and accessible to users.

Why It Exists: Guarantees business continuity and meets Service Level Agreements (SLAs).

REAL WORLD USAGE & TRADEOFF

Usage: High-availability financial gateways targeting 'five nines' (99.999%) uptime.

Trade-off: System availability vs database consistency during network partitions (CAP theorem).

COMMON MISTAKE

Confusing availability with reliability. A slow, buggy system can still be 'available'.

INTERVIEWER TRAP

What is the difference between active-passive and active-active failover?

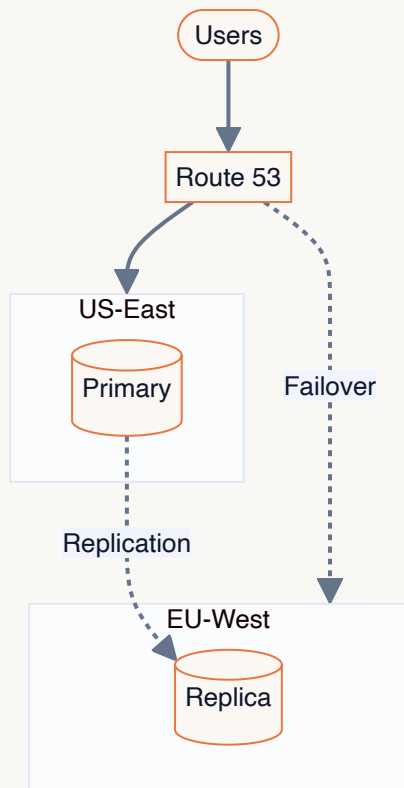
TOP FOLLOW-UP

How do you calculate system availability for serial components?

MEMORY TRICK

Availability is keeping backup generators ready in case the main grid fails.

High Availability Multi-Region ARCHITECTURE



Eraser Diagram Spec

// Nodes

Users [icon: users, color: orange]

Route 53 [icon: compass, color: blue]

Group US-East { Primary [icon: database, color: green] }

Group EU-West { Replica [icon: database, color: green] }

// Edges

Users > Route 53

Route 53 > Primary

Primary > Replica [label: Replication, style: dashed]

Route 53 > Replica [label: Failover, style: dashed, color: red]

Component Breakdown

DNS RoutingDirects traffic to healthy regions.

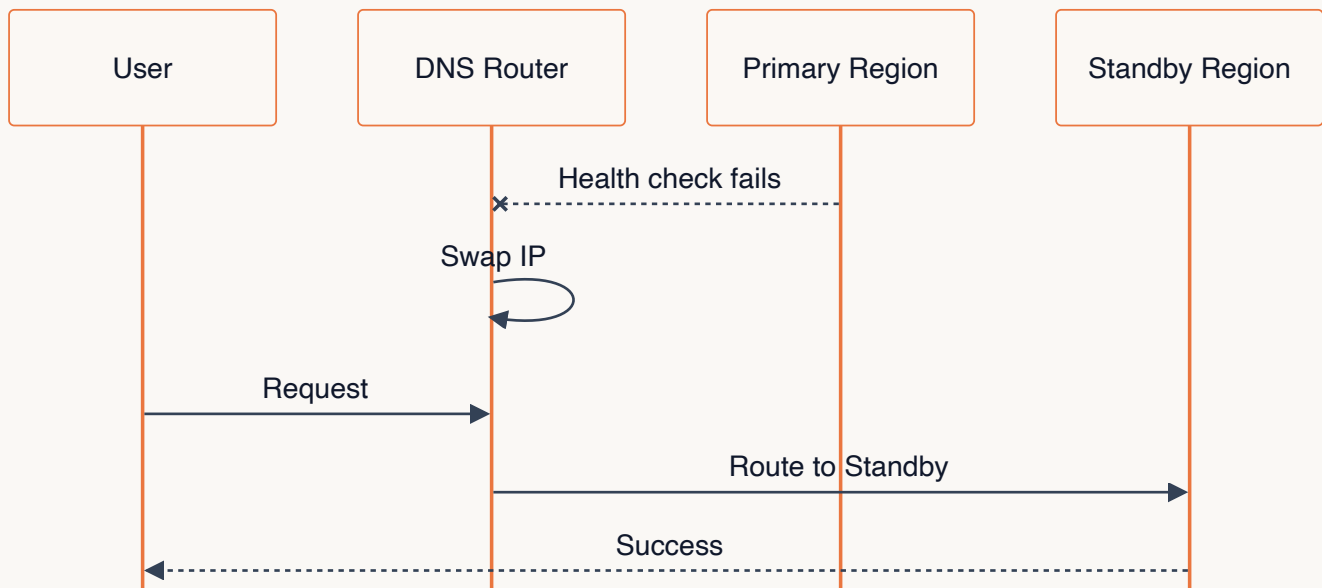
ReplicationMaintains state across geographic regions.

FailoverFallback mechanism during regional outages.

Visual Layout Description

Two distinct geographic clusters separated by a global DNS router.

DNS Failover Sequence REQUEST FLOW



Eraser Diagram Spec

Primary Region > DNS Router: Health check fails [color: red]

DNS Router > DNS Router: Swap IP

User > DNS Router: Request

DNS Router > Standby Region: Route to Standby

Standby Region > User: Success

Component Breakdown

Health Check Periodic ping to verify region uptime.

TTL Swap DNS updates routing IP record.

Promotion Standby DB becomes primary.

Visual Layout Description

Illustrates the failure detection and routing recovery sequence.

Request Flow: Active Region drops -> DNS failover triggers -> Standby Region promoted to active.

Pros: Maintains user trust, reduces transaction losses.

Cons: Requires expensive active-active multi-region replication setups.

Reliability

DEFINITION & WHY IT EXISTS

Definition: The probability that a system performs its required functions under stated conditions for a specified period.

Why It Exists: Ensures calculations are correct and transactions complete successfully.

REAL WORLD USAGE & TRADEOFF

Usage: Banking databases requiring strict transaction completions.

Trade-off: Execution speed (no logs) vs transaction reliability (atomic writes).

COMMON MISTAKE

Believing redundant servers guarantee reliability. Redundancy only supports availability.

INTERVIEWER TRAP

How do you guarantee transaction reliability?

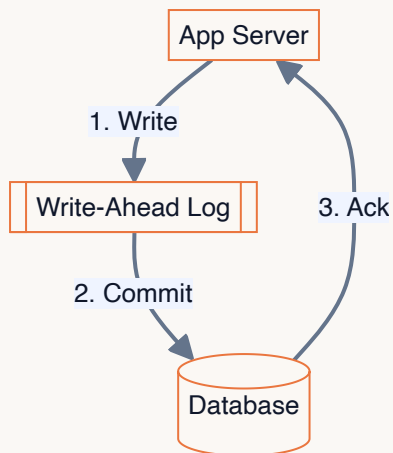
TOP FOLLOW-UP

What is a write-ahead log?

MEMORY TRICK

Reliability is a car that starts every time; availability is having two cars.

Reliability and Write-Ahead Logging ARCHITECTURE



Eraser Diagram Spec

// Nodes

```
App Server [icon: server, color: blue]
Write-Ahead Log [icon: list, color: yellow]
Database [icon: database, color: green]
```

// Edges

```
App Server > Write-Ahead Log [label: 1. Write]
Write-Ahead Log > Database [label: 2. Commit]
Database > App Server [label: 3. Ack]
```

Component Breakdown

Write-Ahead Log Append-only ledger guaranteeing writes.

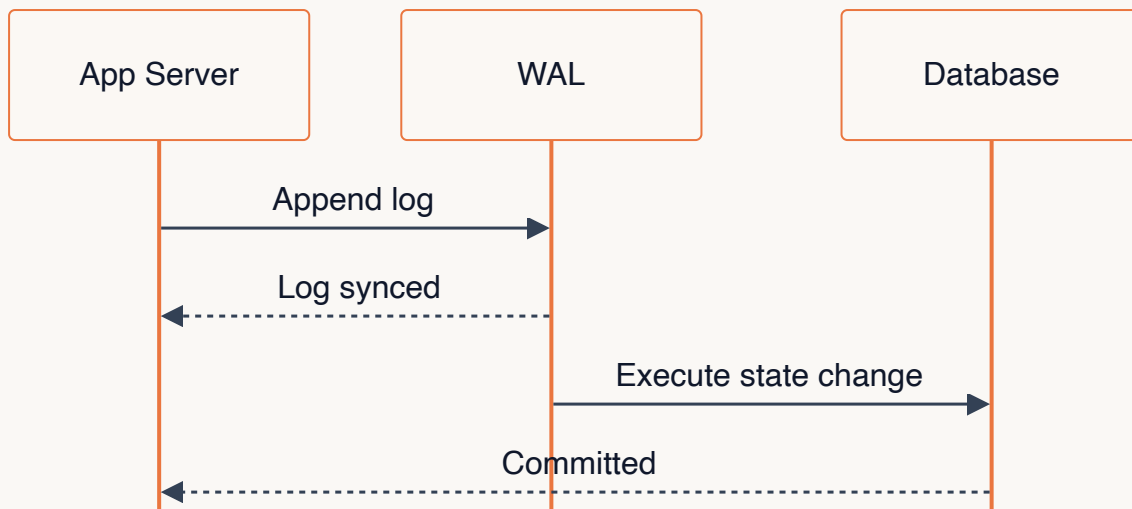
Database Store Final resting place for structured state.

Visual Layout Description

A sequential triad showing the safety buffer between application and storage.

Atomic Commit Sequence

REQUEST FLOW



○ Eraser Diagram Spec

App Server > WAL: Append log
WAL > App Server: Log synced
WAL > Database: Execute state change
Database > App Server: Committed

■ Component Breakdown

Sync Disk Flushing log to physical storage.

Execute Applying change to DB memory structures.

▣ Visual Layout Description

Shows the blocking atomic wait states during a reliable commit.

Request Flow: Input → Log transaction → Execute state change → Return verified output.

Pros: Ensures transaction safety, data accuracy, and user trust.

Cons: Slightly slows down processing due to safety checks.

Fault Tolerance

DEFINITION & WHY IT EXISTS

Definition: A system's ability to continue operating properly in the event of the failure of some of its components.

Why It Exists: Prevents single component failures from bringing down the entire application.

REAL WORLD USAGE & TRADEOFF

Usage: Netflix using Chaos Monkey to terminate servers in production to test resilience.

Trade-off: System component redundancy vs operational infrastructure budget.

COMMON MISTAKE

Assuming failover is zero-latency. Promoting a standby db node takes time.

INTERVIEWER TRAP

What is a single point of failure (SPOF)?

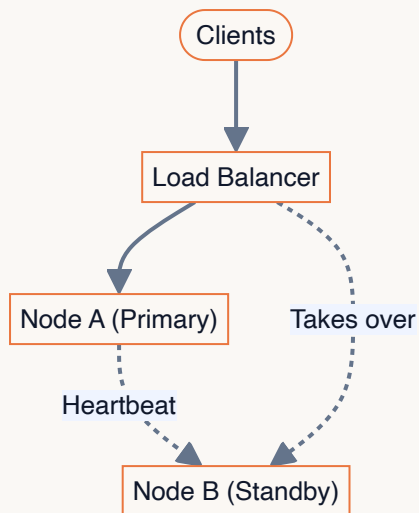
TOP FOLLOW-UP

How does split-brain happen in active-active cluster failovers?

MEMORY TRICK

Fault tolerance is dual tires on semi-trucks: if one pops, the truck keeps moving.

Active-Passive Node Failover ARCHITECTURE



Eraser Diagram Spec

```
// Nodes
Clients [icon: users, color: orange]
Load Balancer [icon: server, color: blue]
Node A [icon: box, color: blue]
Node B [icon: box, color: blue]

// Edges
Clients > Load Balancer
Load Balancer > Node A
Node A > Node B [label: Heartbeat, style: dashed]
Load Balancer > Node B [label: Failover, style: dashed, color: red]
```

Component Breakdown

Primary Node Handles active connections.

Standby Node Idles until failure detection.

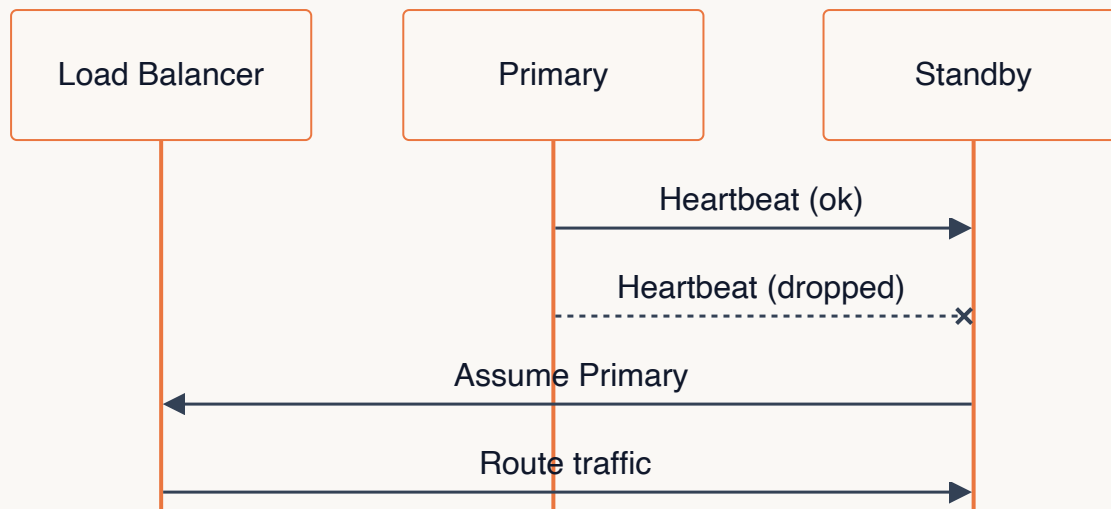
Heartbeat Pulse check between nodes.

Visual Layout Description

Parallel active and passive nodes connected via a heartbeat bridge.

Failover Recovery Sequence

REQUEST FLOW



Eraser Diagram Spec

```
Primary > Standby: Heartbeat (ok)
Primary > Standby: Heartbeat (dropped) [color: red]
Standby > Load Balancer: Assume Primary
Load Balancer > Standby: Route traffic
```

Component Breakdown

Detection Standby notices missing pings.

Assumption Standby elevates privileges.

Re-route LB updates target group.

Visual Layout Description

Chronological timeline of a node death and subsequent takeover.

Request Flow: Primary fails -> Standby detects loss of heartbeat -> Standby assumes primary role.

Pros: Prevents total system outages, enabling graceful degradation.

Cons: Requires hardware redundancy, increasing costs.

Latency

DEFINITION & WHY IT EXISTS

Definition: The time it takes for a data packet to travel from source to destination.

Why It Exists: Directly impacts user experience and interface responsiveness.

REAL WORLD USAGE & TRADEOFF

Usage: High-frequency trading systems where millisecond delays cause lost transactions.

Trade-off: Deep computation checks (higher latency) vs rapid cached returns.

COMMON MISTAKE

Believing network bandwidth solves latency. Latency is limited by the speed of light.

INTERVIEWER TRAP

How do you reduce API latency?

TOP FOLLOW-UP

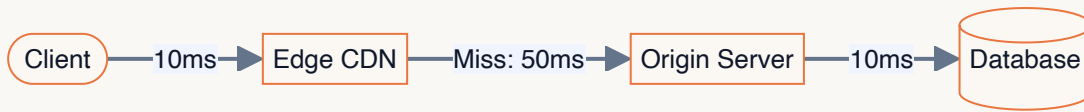
What is the difference between latency and round-trip time (RTT)?

MEMORY TRICK

Latency is the time spent waiting for your food order to arrive at your table.

CDN Edge Caching Architecture

ARCHITECTURE



◇ Eraser Diagram Spec

// Nodes

Client [icon: smartphone, color: orange]

Edge CDN [icon: server, color: blue]

Origin Server [icon: server, color: blue]

Database [icon: database, color: green]

// Edges

Client > Edge CDN [label: 10ms]

Edge CDN > Origin Server [label: Cache Miss, style: dashed]

Origin Server > Database [label: 10ms]

■ Component Breakdown

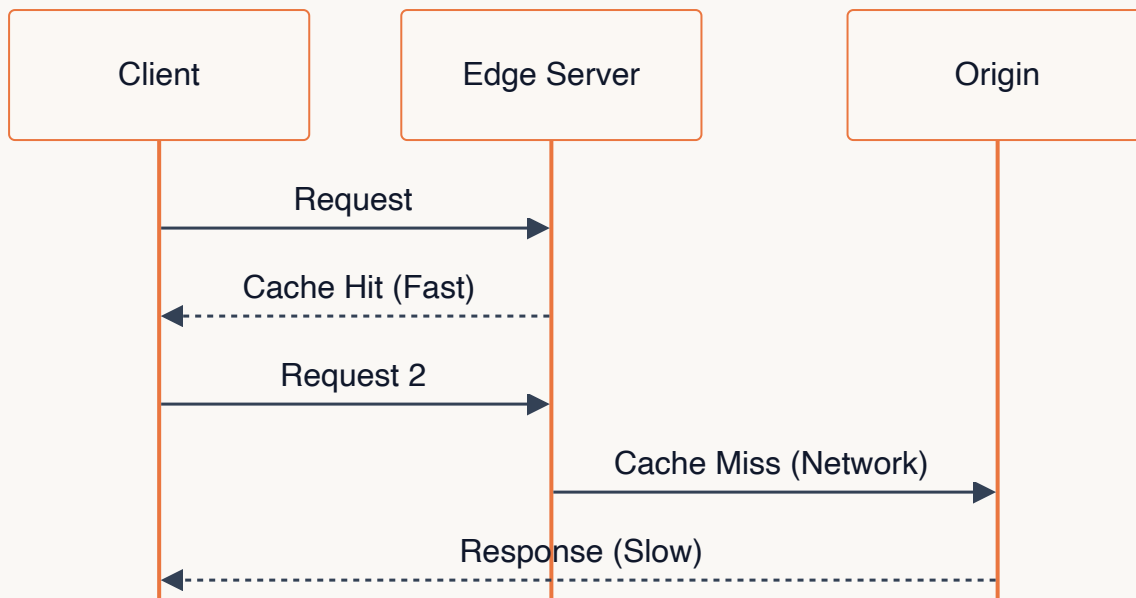
Edge NodeGeographically close caching server.

OriginSource of truth handling dynamic requests.

□ Visual Layout Description

Linear progression demonstrating physical distance impacts on network time.

Cache Miss Sequence REQUEST FLOW



Eraser Diagram Spec

Client > Edge Server: Request
Edge Server > Client: Cache Hit (Fast) [color: green]
Client > Edge Server: Request 2
Edge Server > Origin: Cache Miss (Network) [color: red]
Origin > Client: Response (Slow)

Component Breakdown

Cache Hit Instant return from memory.

Cache Miss Blocking network trip to origin server.

Visual Layout Description

Contrasts the immediate turnaround of a hit vs the multi-hop delay of a miss.

Request Flow: Network transmission → Server processing → Network transmission back.

Pros: Improves page load speeds and user retention.

Cons: Minimizing latency requires expensive CDNs and caching architectures.

Throughput

DEFINITION & WHY IT EXISTS

Definition: The number of requests or data transactions processed by a system within a given time frame.

Why It Exists: Measures total processing capacities under heavy loads.

REAL WORLD USAGE & TRADEOFF

Usage: Data streaming platforms measuring operations processed per second (RPS).

Trade-off: High throughput (batch processing) vs low latency (instant single returns).

COMMON MISTAKE

Confusing throughput with speed. A pipeline can move tons of water slowly.

INTERVIEWER TRAP

How do you increase database throughput?

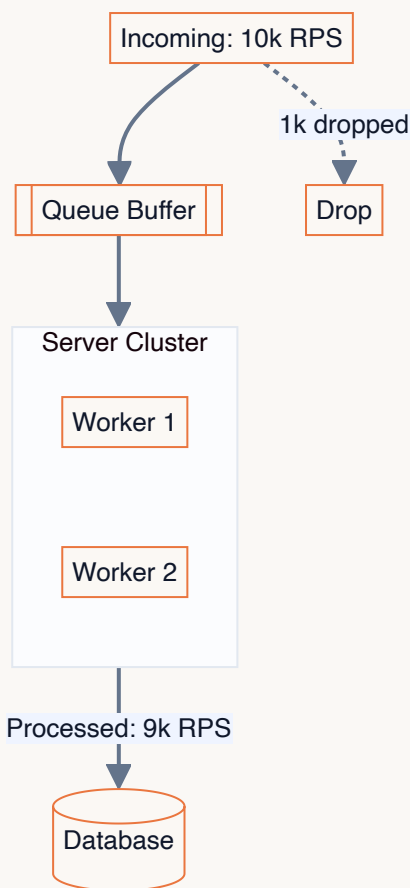
TOP FOLLOW-UP

How do latency and throughput relate?

MEMORY TRICK

Throughput is the width of the highway; latency is the speed limit.

High Throughput Batching ARCHITECTURE



Eraser Diagram Spec

```
// Nodes
Incoming [icon: download, color: orange]
Queue Buffer [icon: list, color: yellow]
Group Server Cluster {
  Worker 1 [icon: box, color: blue]
  Worker 2 [icon: box, color: blue]
}
Database [icon: database, color: green]
```

```
// Edges
Incoming > Queue Buffer
Queue Buffer > Worker 1
Queue Buffer > Worker 2
Worker 1 > Database
Worker 2 > Database
```

Component Breakdown

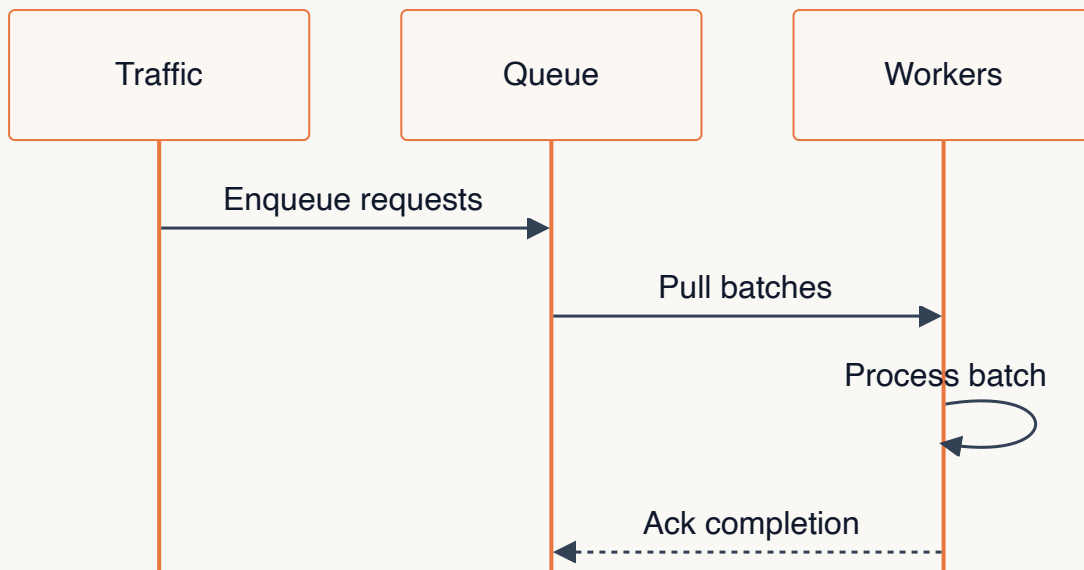
BufferAbsorbs immediate volume.

Worker PoolParallel processors draining queue.

Visual Layout Description

Funnel architecture showing massive input converging through parallel processors.

Parallel Queue Draining REQUEST FLOW



○ Eraser Diagram Spec

Traffic > Queue: Enqueue requests

Queue > Workers: Pull batches

Workers > Workers: Process batch

Workers > Queue: Ack completion

■ Component Breakdown

Batching Grouping records for DB efficiency.

Parallelism Simultaneous execution threads.

▣ Visual Layout Description

Shows asynchronous ingestion disconnected from processing speeds.

Request Flow: Requests queue in buffer → Cluster processes tasks in parallel → Metric recorded.

Pros: Enables processing high data volumes, supporting bulk transactions.

Cons: High throughput systems can saturate network switches and memory buses.

CAP Theorem

DEFINITION & WHY IT EXISTS

Definition: States that a distributed data store can simultaneously provide at most two of: Consistency, Availability, and Partition tolerance.

Why It Exists: Provides a framework for understanding tradeoffs in distributed databases.

REAL WORLD USAGE & TRADEOFF

Usage: Choosing Cassandra (AP) for high write speeds, or Postgres (CP) for transactions.

Trade-off: Rejecting updates (Consistency) vs serving outdated data (Availability).

COMMON MISTAKE

Believing partition tolerance is optional. Network drops WILL happen in distributed systems.

INTERVIEWER TRAP

What happens in an AP system during a network partition?

TOP FOLLOW-UP

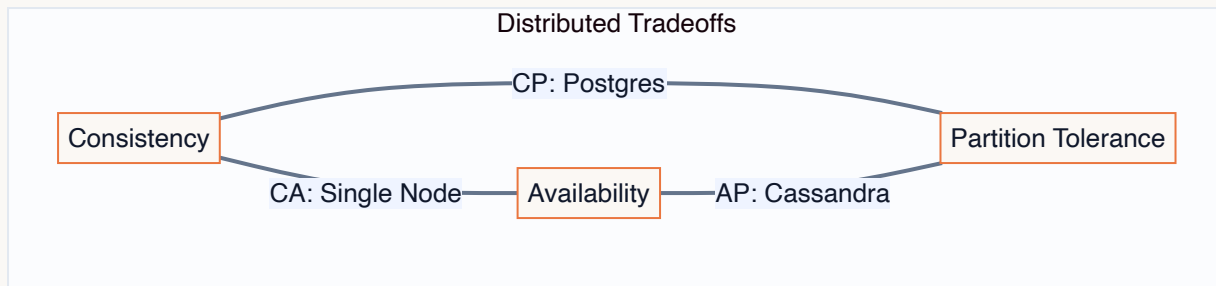
Can a distributed database avoid Partitions (P)?

MEMORY TRICK

CAP is choosing two: Fast delivery, Cheap price, or High quality.

CAP Theorem Tradeoffs

ARCHITECTURE



Eraser Diagram Spec

// Nodes

Consistency [icon: check-circle, color: blue]

Availability [icon: activity, color: green]

Partition Tolerance [icon: alert-triangle, color: red]

// Edges

Consistency - Partition Tolerance [label: CP]

Availability - Partition Tolerance [label: AP]

Consistency - Availability [label: CA]

Component Breakdown

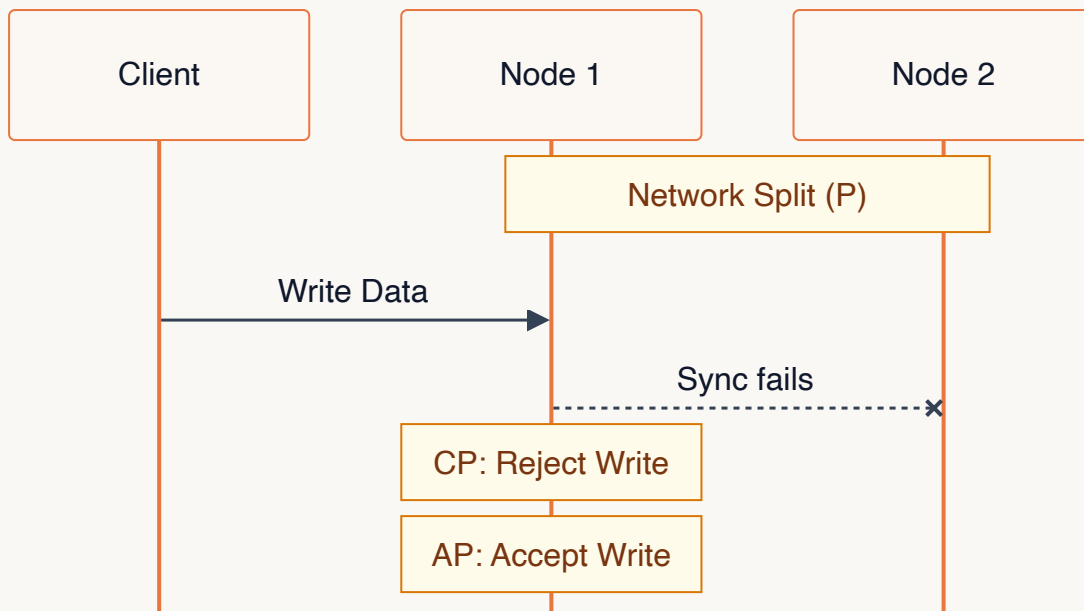
Consistency All reads receive the most recent write.

Availability Every request receives a valid response.

Visual Layout Description

Triangle matrix showing the impossibility of achieving all three vertices.

Network Partition Resolution REQUEST FLOW



○ Eraser Diagram Spec

Client > Node 1: Write Data
 Node 1 > Node 2: Sync fails [color: red]
 Node 1 > Client: CP: Reject Write [color: red]
 Node 1 > Client: AP: Accept Write [color: green]

■ Component Breakdown

Network Split Communication break between nodes.

Reject System stops to maintain accuracy.

▣ Visual Layout Description

Demonstrates the forced choice when a network line is severed.

Request Flow: Network Split -> Choose to reject write (CP) OR write and allow drift (AP).

Pros: Guides database technology selection for specific applications.

Cons: Forces choosing between absolute consistency and constant availability.

Consistency Models

DEFINITION & WHY IT EXISTS

Definition: Rules governing how database updates become visible to parallel read requests.

Why It Exists: Balances read latencies against data synchronization constraints.

REAL WORLD USAGE & TRADEOFF

Usage: Social media feeds using eventual consistency, while banks require strong consistency.

Trade-off: Strict data accuracy (Strong) vs low-latency write performance (Eventual).

COMMON MISTAKE

Assuming eventual consistency syncs within milliseconds under heavy network partition splits.

INTERVIEWER TRAP

What is read-after-write consistency?

TOP FOLLOW-UP

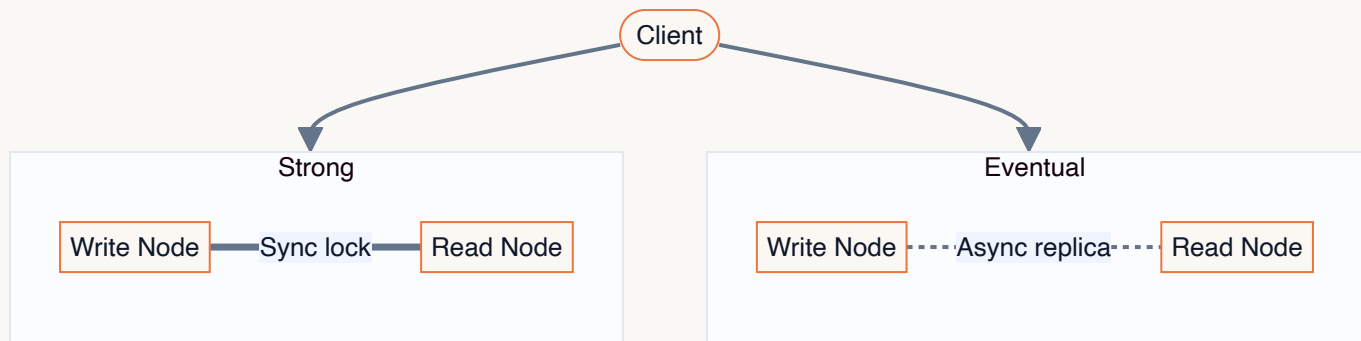
What is linearizability?

MEMORY TRICK

Strong consistency is group chat calls; eventual consistency is sending letters.

Strong vs Eventual Consistency

ARCHITECTURE



Eraser Diagram Spec

```
// Nodes
Client [icon: user, color: orange]
Group Strong {
  W1 [icon: edit, color: blue]
  R1 [icon: eye, color: blue]
}
Group Eventual {
  W2 [icon: edit, color: green]
  R2 [icon: eye, color: green]
}

// Edges
Client > W1
Client > W2
W1 > R1 [label: Sync lock]
W2 > R2 [label: Async replica, style: dashed]
```

Component Breakdown

Sync Lock Blocks returns until replication completes.

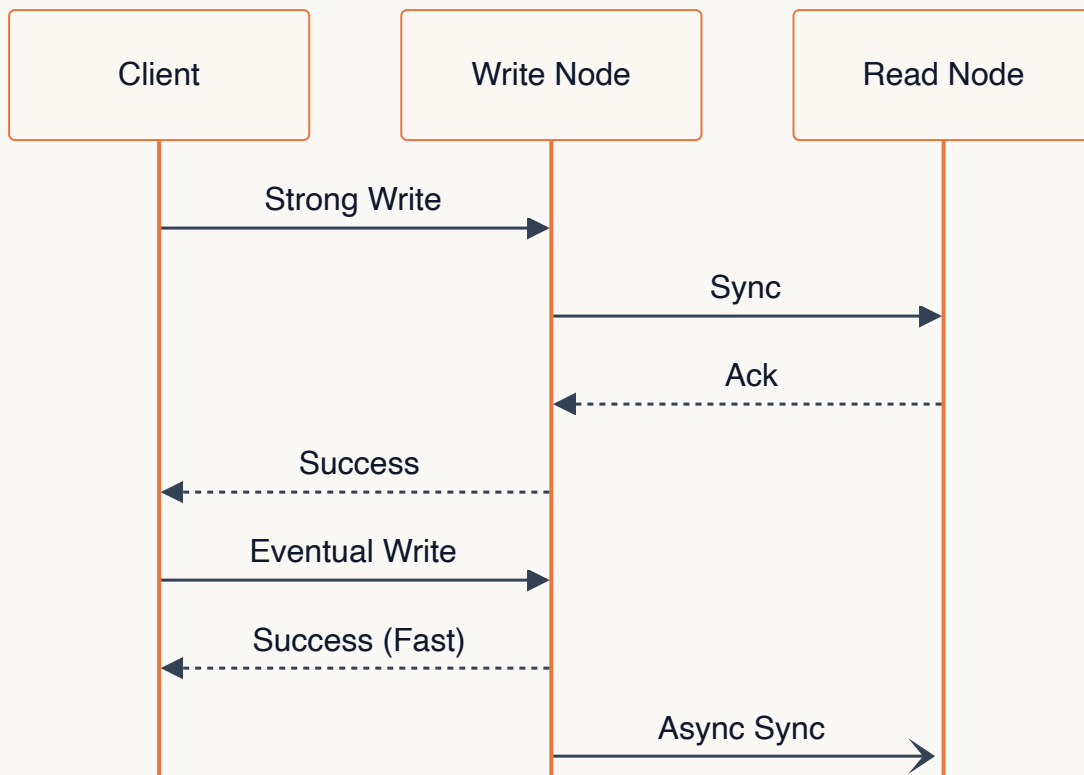
Async Replica Returns immediately, replicates later.

Visual Layout Description

Side-by-side comparison of synchronous blocking vs asynchronous background tasks.

Consistency Timing

REQUEST FLOW



Eraser Diagram Spec

```

Client > Write Node: Strong Write
Write Node > Read Node: Sync
Read Node > Write Node: Ack
Write Node > Client: Success
Client > Write Node: Eventual Write
Write Node > Client: Success (Fast) [color: green]
Write Node > Read Node: Async Sync [style: dashed]
  
```

Component Breakdown

Acknowledgment Confirmation of safely written bytes.

Fast Return Skipping the ack wait for speed.

Visual Layout Description

Timeline emphasizing the extra latency penalty paid for strong data accuracy.

Request Flow: Strong: Write blocks until all replicas confirm. Eventual: Return success instantly, sync in background.

Pros: Strong: Reads always return latest data. Eventual: Low latency writes.

Cons: Strong: High latency writes. Eventual: Reads can return stale data.

Horizontal vs Vertical Scaling

DEFINITION & WHY IT EXISTS

Definition: Scaling horizontally adds more machines to a system pool; scaling vertically upgrades existing server resources.
Why It Exists: Defines the hardware scaling roadmap for growth.

REAL WORLD USAGE & TRADEOFF

Usage: Upgrading a DB server CPU (Vertical) vs adding API nodes (Horizontal).
Trade-off: Simplicity (Vertical) vs scalability limits and high availability (Horizontal).

COMMON MISTAKE

Assuming horizontal scaling is free of architectural changes. Application servers must be stateless.

INTERVIEWER TRAP

When should you scale vertically?

TOP FOLLOW-UP

How does sharding enable horizontal database scaling?

MEMORY TRICK

Vertical is buying a bigger truck; horizontal is hiring a fleet of trucks.

Scaling Vectors ARCHITECTURE



Eraser Diagram Spec

```
// Nodes
Group Vertical {
  2 CPU [icon: server, color: blue]
  64 CPU [icon: database, color: blue]
}
Group Horizontal {
  Server 1 [icon: box, color: green]
  Server 2 [icon: box, color: green]
  Server 3 [icon: box, color: green]
}

// Edges
2 CPU > 64 CPU
Server 1 > Server 2 [style: dashed]
Server 2 > Server 3 [style: dashed]
```

Component Breakdown

VerticalHardware upgrades to existing boxes.

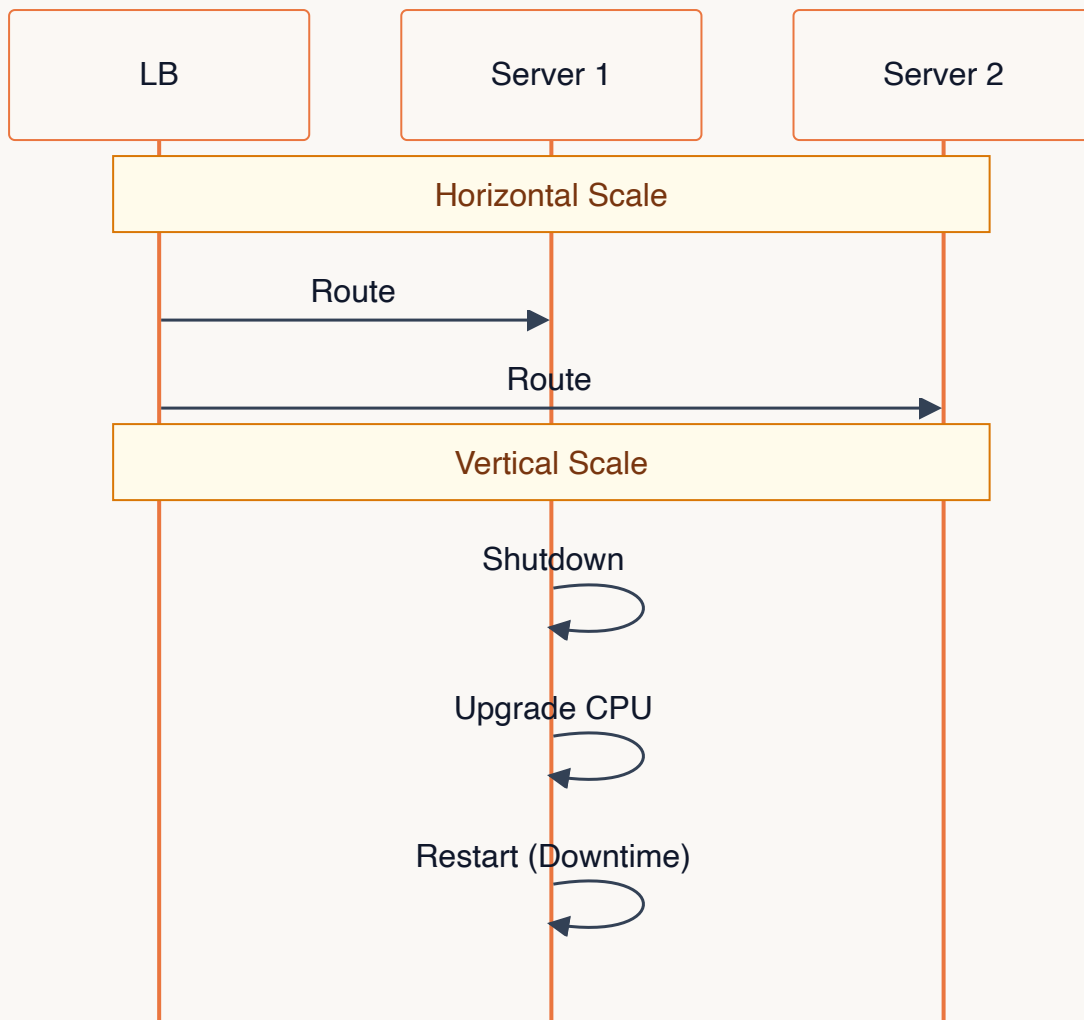
HorizontalAdding identical boxes to a pool.

Visual Layout Description

Contrasts a single growing entity against a replicating swarm.

Downtime vs Auto-Scale

REQUEST FLOW



Eraser Diagram Spec

```
LB > Server 1: Route
LB > Server 2: Route
Server 1 > Server 1: Shutdown [color: red]
Server 1 > Server 1: Upgrade CPU
Server 1 > Server 1: Restart (Downtime) [color: red]
```

Component Breakdown

Downtime Vertical scaling requires reboots.

Routing Horizontal scaling uses LB pools.

Visual Layout Description

Shows the operational disruption of vertical upgrades vs seamless horizontal scaling.

Request Flow: Vertical: Shutdown → Upgrade → Restart. Horizontal: Spin up node → Register with Load Balancer.

Pros: Vertical: Zero code change. Horizontal: Infinite scale ceiling.

Cons: Vertical: Strict hardware limits. Horizontal: Complex state sync requirements.

Load Balancer

DEFINITION & WHY IT EXISTS

Definition: A device or service that distributes network traffic across a cluster of servers.

Why It Exists: Prevents single server overloads and ensures high availability.

REAL WORLD USAGE & TRADEOFF

Usage: AWS ALB routing web requests to healthy target group instances.

Trade-off: Layer 4 routing (fast, pure TCP) vs Layer 7 routing (slower, path-based parsing).

COMMON MISTAKE

Neglecting health check configurations. Poor health checks route traffic to crashed nodes.

INTERVIEWER TRAP

What is SSL termination at the load balancer?

TOP FOLLOW-UP

Explain round-robin vs least-connections algorithms.

MEMORY TRICK

A load balancer is a traffic cop directing cars to open highway lanes.

Reverse Proxy

DEFINITION & WHY IT EXISTS

Definition: A server that sits in front of backend servers, routing client requests to them.

Why It Exists: Protects backend servers, manages SSL, caches static files, and compresses traffic.

REAL WORLD USAGE & TRADEOFF

Usage: Nginx shielding internal Node.js backend ports from external access.

Trade-off: Centralized security configurations vs single point of failure risks.

COMMON MISTAKE

Exposing internal backend ports directly to the internet, bypassing proxy protections.

INTERVIEWER TRAP

What is the difference between a forward proxy and a reverse proxy?

TOP FOLLOW-UP

How does Nginx handle static file caching?

MEMORY TRICK

A reverse proxy is a receptionist who receives all packages and forwards them to workers.

CDN

DEFINITION & WHY IT EXISTS

Definition: Content Delivery Network: a geographically distributed group of servers caching content near users.

Why It Exists: Minimizes page load latency by serving static assets from edge locations.

REAL WORLD USAGE & TRADEOFF

Usage: Cloudflare caching website images, videos, and CSS stylesheets globally.

Trade-off: Immediate asset deployment (no CDN) vs edge caching speed (requires cache invalidation).

COMMON MISTAKE

Caching dynamic HTML pages that contain user-specific session data on edge nodes.

INTERVIEWER TRAP

How does CDN cache invalidation work?

TOP FOLLOW-UP

What is a push vs pull CDN configuration?

MEMORY TRICK

A CDN is like having local warehouses instead of shipping every package from headquarters.

API Gateway

DEFINITION & WHY IT EXISTS

Definition: A single entry point for client requests, managing routing, auth, rate limiting, and metrics gathering.

Why It Exists: Simplifies client integration by centralizing microservices access.

REAL WORLD USAGE & TRADEOFF

Usage: Kong or AWS API Gateway routing requests to internal microservice nodes.

Trade-off: Decoupled microservice security vs centralized gateway processing overheads.

COMMON MISTAKE

Placing heavy business processing logic inside the API gateway configuration layer.

INTERVIEWER TRAP

How does an API gateway handle service rate limiting?

TOP FOLLOW-UP

Explain API gateway routing vs reverse proxy routing.

MEMORY TRICK

An API gateway is the security desk at a corporate building: check ID and route visitors.

Service Discovery

DEFINITION & WHY IT EXISTS

Definition: The automatic detection of devices and services offered by a computer network.

Why It Exists: Allows microservices to locate and connect with each other as IP addresses dynamically scale.

REAL WORLD USAGE & TRADEOFF

Usage: Consul or Eureka tracking running instances of payment and catalog services.

Trade-off: Dynamic registry lookups (Consul) vs static configuration bindings.

COMMON MISTAKE

Hardcoding IP addresses in microservice config files instead of using discovery names.

INTERVIEWER TRAP

What is client-side vs server-side service discovery?

TOP FOLLOW-UP

How do heartbeat checks maintain registry integrity?

MEMORY TRICK

Service discovery is the yellow pages: lookup contact info before calling.

DNS Flow

DEFINITION & WHY IT EXISTS

Definition: The process of resolving human-readable domain names into machine-readable IP addresses.

Why It Exists: Enables web browsers to locate and connect to servers hosting websites.

REAL WORLD USAGE & TRADEOFF

Usage: Resolving domain 'grindos.pranavx.in' to IP address '192.0.2.1'.

Trade-off: Short DNS TTL (fast updates, high query load) vs long DNS TTL (slow updates, cached).

COMMON MISTAKE

Setting long DNS TTLs during server migration windows, delaying IP updates.

INTERVIEWER TRAP

What is the difference between recursive and authoritative DNS servers?

TOP FOLLOW-UP

What happens when DNS cache poison attacks succeed?

MEMORY TRICK

DNS is the contact contact list on your phone: lookup 'Mom' to call her number.

Caching Basics

DEFINITION & WHY IT EXISTS

Definition: A hardware or software component that stores data so future requests can be served faster.

Why It Exists: Reduces read latencies and offloads database search compute loads.

REAL WORLD USAGE & TRADEOFF

Usage: Redis caching user session profiles in application memory.

Trade-off: Immediate database queries (accurate, slow) vs cached memory lookups (stale risk, fast).

COMMON MISTAKE

Neglecting to set TTLs (Time-to-Live) on cache keys, causing memory bloat.

INTERVIEWER TRAP

Explain write-through vs write-behind caching strategies.

TOP FOLLOW-UP

What are cache eviction policies like LRU and LFU?

MEMORY TRICK

Caching is keeping your frequently used tools on your desk instead of in the toolbox.

SQL vs NoSQL

DEFINITION & WHY IT EXISTS

Definition: SQL databases are relational and structured; NoSQL databases are non-relational and schema-less.

Why It Exists: Guides database technology selection based on data models and scaling needs.

REAL WORLD USAGE & TRADEOFF

Usage: Postgres (SQL) for financial ledgers; MongoDB (NoSQL) for user profiles.

Trade-off: Relational query safety (SQL) vs schema flexibility and horizontal scale capabilities (NoSQL).

COMMON MISTAKE

Using NoSQL when your data models require deep relational queries and foreign key safety.

INTERVIEWER TRAP

Explain ACID compliance in relational databases.

TOP FOLLOW-UP

How do NoSQL databases scale horizontally?

MEMORY TRICK

SQL is an organized file cabinet; NoSQL is a storage box where you drop items.

Database Indexing

DEFINITION & WHY IT EXISTS

Definition: A database structure that improves the speed of data retrieval operations on a table.

Why It Exists: Transforms slow full-table scans into fast index seeks.

REAL WORLD USAGE & TRADEOFF

Usage: Creating a B-Tree index on a SQL `user\_id` column to speed lookups.

Trade-off: Faster read performance vs slower write queries and index storage footprints.

COMMON MISTAKE

Indexing every single column. This bloats storage and degrades write performance.

INTERVIEWER TRAP

How does a B-Tree index work under the hood?

TOP FOLLOW-UP

What is the difference between a clustered and non-clustered index?

MEMORY TRICK

An index is the index at the back of a textbook: lookup page numbers directly.

Replication

DEFINITION & WHY IT EXISTS

Definition: The practice of copying data across multiple database servers to ensure reliability and accessibility.

Why It Exists: Protects against database server hardware failures and data loss.

REAL WORLD USAGE & TRADEOFF

Usage: Replicating primary DB transactions to a hot standby standby server.

Trade-off: Synchronous replication (safe, high latency) vs asynchronous replication (fast, data loss risk).

COMMON MISTAKE

Assuming replication is a backup. If you run a bad DELETE query, it propagates to all replicas.

INTERVIEWER TRAP

What is the difference between master-slave and multi-master replication?

TOP FOLLOW-UP

Explain replication lag issues.

MEMORY TRICK

Replication is carbon copying a document: one copy goes in the desk, one in the safe.

Read Replicas

DEFINITION & WHY IT EXISTS

Definition: Database server copies dedicated to serving read queries, offloading the primary database.

Why It Exists: Increases system read capacities, allowing scaling of read-heavy systems.

REAL WORLD USAGE & TRADEOFF

Usage: Directing user dashboard loads to read replicas, reserving write queries for the primary.

Trade-off: Strict read consistency vs scaled read query throughput capacity.

COMMON MISTAKE

Sending write transactions to read replicas. Replicas are read-only and reject updates.

INTERVIEWER TRAP

How do you handle read-after-write consistency with read replicas?

TOP FOLLOW-UP

What happens if a read replica crashes?

MEMORY TRICK

A read replica is printing multiple copies of a menu so all customers can read at once.

Sharding

DEFINITION & WHY IT EXISTS

Definition: A database partitioning technique that splits a single database across multiple server instances.

Why It Exists: Enables horizontal database scaling when single servers hit CPU/storage limits.

REAL WORLD USAGE & TRADEOFF

Usage: Sharding users by hash ID: Users 1–10k on DB 1; Users 10k–20k on DB 2.

Trade-off: Single DB server simplicity vs distributed shard database scale capacities.

COMMON MISTAKE

Sharding databases prematurely. It introduces massive query complexity that is hard to revert.

INTERVIEWER TRAP

What is a hot shard and how do you resolve it?

TOP FOLLOW-UP

Explain range-based vs hash-based sharding.

MEMORY TRICK

Sharding is splitting a massive book index into separate alphabetical volume binders.

Partitioning

DEFINITION & WHY IT EXISTS

Definition: Splitting database tables into smaller logical parts within the same database server.

Why It Exists: Improves query speeds and maintenance overheads on large tables.

REAL WORLD USAGE & TRADEOFF

Usage: Partitioning an invoice table by year: `invoices\_2025`, `invoices\_2026`.

Trade-off: Table query simplicity vs table partition file size maintenance.

COMMON MISTAKE

Partitioning tables that have low row counts. This adds directory traversal overheads.

INTERVIEWER TRAP

What is the difference between horizontal and vertical partitioning?

TOP FOLLOW-UP

How does range partitioning differ from list partitioning?

MEMORY TRICK

Partitioning is sorting files into folders by year inside a single cabinet drawer.

CQRS

DEFINITION & WHY IT EXISTS

Definition: Command Query Responsibility Segregation: separating read models from write models.

Why It Exists: Optimizes query performance and scaling by isolating write schemas from read indexes.

REAL WORLD USAGE & TRADEOFF

Usage: Using an event store for writes, replicating to Elasticsearch for searches.

Trade-off: Strict data synchronization (single DB) vs scaled query performance (decoupled CQRS).

COMMON MISTAKE

Implementing CQRS on simple CRUD applications. It adds unnecessary infrastructure overhead.

INTERVIEWER TRAP

How do you handle sync lag in CQRS architecture?

TOP FOLLOW-UP

What is event sourcing and how does it relate to CQRS?

MEMORY TRICK

CQRS is a restaurant kitchen: one group prepares food (write), a waiter delivers plates (read).

Eventual Consistency

DEFINITION & WHY IT EXISTS

Definition: A consistency model where database replicas eventually sync updates if no new modifications occur.

Why It Exists: Allows distributed databases to scale writes without blocking for sync loops.

REAL WORLD USAGE & TRADEOFF

Usage: Social media likes updates: numbers sync across global user views within seconds.

Trade-off: Immediate update accuracy (Strong) vs fast write speeds (Eventual).

COMMON MISTAKE

Relying on eventual consistency for bank account transactions, causing overdraft conflicts.

INTERVIEWER TRAP

How does dynamo-style DB handle eventual consistency?

TOP FOLLOW-UP

Explain conflict resolution (LWW vs CRDT) in eventual consistency.

MEMORY TRICK

Eventual consistency is sending mail: updates arrive at different times but eventually sync.

Distributed Transactions

DEFINITION & WHY IT EXISTS

Definition: Transactions that update tables across multiple distributed database nodes.

Why It Exists: Enables transaction consistency (ACID) across independent databases.

REAL WORLD USAGE & TRADEOFF

Usage: A payment service debiting balance in DB 1, while orders DB 2 records purchases.

Trade-off: Distributed data consistency (2PC) vs system performance and write availability.

COMMON MISTAKE

Relying on 2PC over slow WAN connections, causing transaction timeouts and thread lockups.

INTERVIEWER TRAP

Explain the Two-Phase Commit (2PC) protocol.

TOP FOLLOW-UP

How does 3PC resolve the coordinator block limitation of 2PC?

MEMORY TRICK

2PC is planning a trip with friends: everyone must say 'Yes' before booking tickets.

Message Queues

DEFINITION & WHY IT EXISTS

Definition: A queue structure buffering messages asynchronously between sender and receiver processes.

Why It Exists: Decouples services, manages spikes, and schedules background tasks.

REAL WORLD USAGE & TRADEOFF

Usage: Pushing image upload paths to RabbitMQ for background resizing tasks.

Trade-off: Synchronous API execution (instant confirmation) vs asynchronous background queues.

COMMON MISTAKE

Neglecting worker scale configurations, causing queue backlogs to grow indefinitely.

INTERVIEWER TRAP

What is the difference between push vs pull worker message queues?

TOP FOLLOW-UP

Explain message delivery guarantees (At-least-once, At-most-once).

MEMORY TRICK

A message queue is the inbox tray on a desk: pile up letters to process in order.

Kafka

DEFINITION & WHY IT EXISTS

Definition: A distributed, partitioned, replicated commit log service optimized for high-volume streaming.

Why It Exists: Enables processing millions of telemetry events per second with high durability.

REAL WORLD USAGE & TRADEOFF

Usage: Streaming vehicle coordinate updates at Uber to calculate dynamic routes.

Trade-off: Complex log replay setups (Kafka) vs simple memory queue networks (RabbitMQ).

COMMON MISTAKE

Using Kafka for simple task scheduling where RabbitMQ can handle requirements.

INTERVIEWER TRAP

How does Kafka achieve high write performance?

TOP FOLLOW-UP

Explain partition counts in consumer scaling groups.

MEMORY TRICK

Kafka is a security tape recording: continuously record events to review or replay.

RabbitMQ

DEFINITION & WHY IT EXISTS

Definition: A messaging broker implementing Advanced Message Queuing Protocol (AMQP).

Why It Exists: Enables complex routing logic between microservices.

REAL WORLD USAGE & TRADEOFF

Usage: Routing notifications to email/SMS queues using exchange routing keys.

Trade-off: Advanced routing configurations vs large message backlog storage limits.

COMMON MISTAKE

Failing to configure Dead Letter Exchanges, losing failed message payloads.

INTERVIEWER TRAP

Explain Direct, Fanout, and Topic exchange routing types.

TOP FOLLOW-UP

How do publisher confirms ensure message delivery?

MEMORY TRICK

RabbitMQ is the post office: sort letters by address and distribute to carrier routes.

Pub/Sub

DEFINITION & WHY IT EXISTS

Definition: A messaging pattern where publishers categorise messages into classes, without knowledge of subscribers.

Why It Exists: Enables real-time event broadcasting to multiple subscribers simultaneously.

REAL WORLD USAGE & TRADEOFF

Usage: Broadcasting stock price changes to user terminal screen windows.

Trade-off: Ephemeral live broadcasting (Pub/Sub) vs persistent message queue storage.

COMMON MISTAKE

Relying on raw Pub/Sub for critical transactions that require guaranteed delivery.

INTERVIEWER TRAP

How do you implement persistent delivery patterns in Pub/Sub networks?

TOP FOLLOW-UP

What is the difference between point-to-point and pub/sub messaging?

MEMORY TRICK

Pub/Sub is a radio broadcast: tune in to listen; miss it if you turn off the radio.

Event Driven Architecture

DEFINITION & WHY IT EXISTS

Definition: A software architecture pattern where services respond to state change events.

Why It Exists: Enables loose coupling and asynchronous scaling across microservices.

REAL WORLD USAGE & TRADEOFF

Usage: Checkout triggers 'OrderPlaced' event; inventory and logistics react.

Trade-off: Asynchronous service autonomy vs synchronous transactional consistency.

COMMON MISTAKE

Creating circular event dependency loops, where Event A triggers Event B, triggering Event A.

INTERVIEWER TRAP

How do you handle distributed trace tracking in event architectures?

TOP FOLLOW-UP

What is eventual consistency handling in event flows?

MEMORY TRICK

Event Driven is a restaurant order slip: kitchen, bar, and billing start processing.

Dead Letter Queue

DEFINITION & WHY IT EXISTS

Definition: A queue holding message payloads that failed to process successfully.

Why It Exists: Prevents faulty payloads from blocking worker thread queues (poison pills).

REAL WORLD USAGE & TRADEOFF

Usage: Storing corrupt JSON payloads in DLQ for manual developer inspection.

Trade-off: Instantly deleting failed messages vs storing them for manual investigation (DLQ).

COMMON MISTAKE

Failing to set alerting thresholds on DLQ sizes, letting failed jobs gather dust.

INTERVIEWER TRAP

What causes a message to become a poison pill?

TOP FOLLOW-UP

How do you configure DLQs in RabbitMQ vs SQS?

MEMORY TRICK

A DLQ is the quarantine zone in a lab: isolate dangerous items for safety.

Retry Patterns

DEFINITION & WHY IT EXISTS

Definition: Design patterns that handle transient service failures by retrying operations.

Why It Exists: Enables applications to recover from brief network or external API drops.

REAL WORLD USAGE & TRADEOFF

Usage: Retrying API calls with exponential backoff on HTTP 503 responses.

Trade-off: Fast failure loops (fail fast) vs resilient retry loops (higher resource locks).

COMMON MISTAKE

Retrying permanent errors (e.g., HTTP 401 Unauthorized), which only drains compute credits.

INTERVIEWER TRAP

Why is adding jitter crucial in exponential backoff algorithms?

TOP FOLLOW-UP

What is the difference between transient and permanent errors?

MEMORY TRICK

Retry is dialing a busy phone number again after waiting a few seconds.

Idempotency

DEFINITION & WHY IT EXISTS

Definition: The property where making an API call multiple times has the same effect as making it once.

Why It Exists: Prevents duplicate charges or transactions on network retry operations.

REAL WORLD USAGE & TRADEOFF

Usage: Payment APIs using idempotency keys: `idempotency\_key = req\_uuid`.

Trade-off: Execution speed (no checks) vs duplicate safety (idempotency checks).

COMMON MISTAKE

Using volatile memory caches to store idempotency keys, risking key losses on reboots.

INTERVIEWER TRAP

How do you design an idempotency layer for REST APIs?

TOP FOLLOW-UP

What database storage setups work best for idempotency keys?

MEMORY TRICK

Idempotency is pressing the elevator call button multiple times: the lift still comes once.

Monolith vs Microservices

DEFINITION & WHY IT EXISTS

Definition: Monoliths package code in a single executable; microservices split logic into independent services.

Why It Exists: Defines system code modularity and team scaling roadmaps.

REAL WORLD USAGE & TRADEOFF

Usage: Starting with a monolith MVP, splitting into microservices as engineering teams grow.

Trade-off: Local speed and simplicity (Monolith) vs service independence and scale capabilities (Microservices).

COMMON MISTAKE

Migrating to microservices before your database schemas and service boundaries are clearly defined.

INTERVIEWER TRAP

What is the database-per-service pattern?

TOP FOLLOW-UP

How do microservices communicate efficiently?

MEMORY TRICK

Monolith is a Swiss army knife; microservices are a toolbox of specialized tools.

Circuit Breaker

DEFINITION & WHY IT EXISTS

Definition: A design pattern that blocks requests to failing services, protecting system resources.

Why It Exists: Prevents failing downstream services from creating cascading system timeouts.

REAL WORLD USAGE & TRADEOFF

Usage: Tripping the payment breaker if payment gateway errors exceed 50% in 1 minute.

Trade-off: Retrying requests (hoping for recovery) vs blocking requests (resilience, instant errors).

COMMON MISTAKE

Setting low timeout values on downstream calls, causing the breaker to open on brief spikes.

INTERVIEWER TRAP

Explain Closed, Open, and Half-Open states in circuit breakers.

TOP FOLLOW-UP

What fallback options work best when a breaker opens?

MEMORY TRICK

A circuit breaker is the fuse box in your house: cuts power to prevent electrical fires.

Service Mesh

DEFINITION & WHY IT EXISTS

Definition: A dedicated infrastructure layer managing service-to-service network communications.

Why It Exists: Offloads networking logic (auth, tracing, routing) from application source code.

REAL WORLD USAGE & TRADEOFF

Usage: Using Istio sidecar proxies (Envoy) to manage communication between Kubernetes pods.

Trade-off: Application-level security configs vs infrastructure-level sidecar proxy overheads.

COMMON MISTAKE

Deploying a service mesh on small clusters. It adds unnecessary management complexity.

INTERVIEWER TRAP

What is the data plane vs control plane in service mesh architectures?

TOP FOLLOW-UP

How does mTLS secure service communication?

MEMORY TRICK

A service mesh is having private security guards route and check all visitors in an office.

Saga Pattern

DEFINITION & WHY IT EXISTS

Definition: A design pattern managing distributed transactions through a sequence of local transactions.

Why It Exists: Enables transactional consistency across microservices without database-level write locks.

REAL WORLD USAGE & TRADEOFF

Usage: Processing orders: charge card → if catalog fails, trigger compensation refund flow.

Trade-off: Distributed consistency (2PC lock) vs eventual transaction consistency (Saga).

COMMON MISTAKE

Believing Sagas are ACID. They only provide eventual consistency, allowing data anomalies.

INTERVIEWER TRAP

Explain choreography-based vs orchestration-based Sagas.

TOP FOLLOW-UP

How do you handle failures in compensating transaction flows?

MEMORY TRICK

Saga is booking a vacation: refund flight ticket if the hotel reservation is rejected.

Distributed Tracing

DEFINITION & WHY IT EXISTS

Definition: A method used to profile and monitor requests as they traverse distributed services.

Why It Exists: Enables debugging latency bottlenecks and error sources across microservices.

REAL WORLD USAGE & TRADEOFF

Usage: Using Jaeger to trace gateway request delays down to specific database queries.

Trade-off: Deep tracing data detail vs performance overheads and log storage costs.

COMMON MISTAKE

Failing to pass trace headers in asynchronous worker queues, breaking trace links.

INTERVIEWER TRAP

How does context propagation work in distributed tracing?

TOP FOLLOW-UP

What is the difference between trace ID and span ID?

MEMORY TRICK

Distributed tracing is GPS tracking a package through all sorting hubs.

Rate Limiting

DEFINITION & WHY IT EXISTS

Definition: Throttling client requests based on specified resource limits.

Why It Exists: Prevents resource starvation, brute-force attacks, and API abuse.

REAL WORLD USAGE & TRADEOFF

Usage: Limiting public APIs to 100 requests per minute per IP address.

Trade-off: Strict system protection vs flexible user query rate caps.

COMMON MISTAKE

Storing rate limits in single-node memory arrays. Autoscaling nodes will not sync limits.

INTERVIEWER TRAP

Explain Token Bucket vs Leaky Bucket algorithms.

TOP FOLLOW-UP

How do you implement distributed rate limiting using Redis?

MEMORY TRICK

Rate limiting is the ticket gate at a theater: only allow one person through at a time.

Object Storage

DEFINITION & WHY IT EXISTS

Definition: A storage architecture that manages data as objects, rather than file hierarchies or database blocks.

Why It Exists: Enables storing petabytes of unstructured files reliably and cheaply.

REAL WORLD USAGE & TRADEOFF

Usage: Storing raw user uploads and profile photos on AWS S3.

Trade-off: Relational block search (DB) vs raw file storage scale capacities (Object Store).

COMMON MISTAKE

Using object storage as an active database, continuously overwriting files.

INTERVIEWER TRAP

What is the difference between object storage and block storage?

TOP FOLLOW-UP

Explain eventual consistency in object storage updates.

MEMORY TRICK

Object storage is checking bags at an airport: hand over your bag and receive a ticket tag.

Blob Storage

DEFINITION & WHY IT EXISTS

Definition: Binary Large Object Storage: a service optimized for storing unstructured binary data.

Why It Exists: Provides storage systems for large binary media files (videos, audio, backups).

REAL WORLD USAGE & TRADEOFF

Usage: Azure Blob Storage housing video assets for a media streaming platform.

Trade-off: Structured block partition indexes vs raw binary storage volumes.

COMMON MISTAKE

Storing sensitive corporate documents in public blob containers, exposing links to search indexers.

INTERVIEWER TRAP

What are block, page, and append blobs?

TOP FOLLOW-UP

How do you secure access to binary storage objects?

MEMORY TRICK

Blob storage is a large storage bin: drop in raw materials without packaging.

File Upload Architecture

DEFINITION & WHY IT EXISTS

Definition: An architecture designed to stream, validate, and store user-uploaded files.

Why It Exists: Saves server compute resources by offloading file transfer work.

REAL WORLD USAGE & TRADEOFF

Usage: Using pre-signed S3 URLs so clients upload files directly to cloud storage.

Trade-off: Direct upload parsing (immediate validation) vs pre-signed direct S3 uploads.

COMMON MISTAKE

Buffering file uploads in web server memory, causing processes to crash.

INTERVIEWER TRAP

How do pre-signed URLs protect upload paths?

TOP FOLLOW-UP

How do you handle multi-part file uploads?

MEMORY TRICK

Pre-signed URL is a delivery pass: allows dropping packages at a warehouse.

Media Processing

DEFINITION & WHY IT EXISTS

Definition: A system architecture that processes, transcodes, and compresses media files.

Why It Exists: Optimizes video delivery across varying network connection speeds.

REAL WORLD USAGE & TRADEOFF

Usage: Transcoding raw video uploads into HLS streams (1080p, 720p, 480p) via workers.

Trade-off: High transcode cost/storage vs low user playback buffering speeds.

COMMON MISTAKE

Processing video transcoding synchronously inside client-facing HTTP routes.

INTERVIEWER TRAP

Explain HLS and DASH streaming protocols.

TOP FOLLOW-UP

How do you optimize background transcode worker queues?

MEMORY TRICK

Media processing is translating a book into different languages for global distribution.

Search Architecture

DEFINITION & WHY IT EXISTS

Definition: A system optimized to index and search unstructured text across databases.

Why It Exists: Delivers fast, ranked query matches that standard databases cannot handle.

REAL WORLD USAGE & TRADEOFF

Usage: Using Elasticsearch to index and search product catalogs with typo-tolerance.

Trade-off: Simple database LIKE scans (slow, synchronous) vs Elasticsearch indexing (fast, sync delay).

COMMON MISTAKE

Treating Elasticsearch as your primary database. It does not support ACID transactions.

INTERVIEWER TRAP

How does an inverted index work?

TOP FOLLOW-UP

How do you sync database writes with Elasticsearch indexes?

MEMORY TRICK

Search index is card catalog drawers in a library: find shelf numbers directly.

Authentication

DEFINITION & WHY IT EXISTS

Definition: The process of verifying the identity of a user or system client.

Why It Exists: Ensures that users are who they claim to be, preventing access spoofing.

REAL WORLD USAGE & TRADEOFF

Usage: Verifying user login credentials using salted password hashes.

Trade-off: User login convenience (MFA disabled) vs strict system security (MFA enabled).

COMMON MISTAKE

Storing raw passwords in plaintext inside databases. Always salt and hash passwords.

INTERVIEWER TRAP

What is the difference between authentication and authorization?

TOP FOLLOW-UP

Explain password hashing algorithms like bcrypt.

MEMORY TRICK

Authentication is showing your passport at border control to prove your identity.

Authorization

DEFINITION & WHY IT EXISTS

Definition: The process of verifying what resources an authenticated user has permission to access.

Why It Exists: Prevents users from modifying or viewing data outside their security scope.

REAL WORLD USAGE & TRADEOFF

Usage: Blocking students from accessing recruiter dashboard API routes.

Trade-off: Fine-grained permissions (high management cost) vs simple admin-user roles.

COMMON MISTAKE

Neglecting backend authorization. Always verify roles on the server, not just on the UI.

INTERVIEWER TRAP

What is the principle of least privilege?

TOP FOLLOW-UP

How do you enforce authorization in microservice systems?

MEMORY TRICK

Authorization is using a hotel key card: proves you can enter room 204, but not room 305.

JWT

DEFINITION & WHY IT EXISTS

Definition: JSON Web Token: an open standard for securely transmitting information between parties as a JSON object.

Why It Exists: Enables stateless user session authentication across microservice clusters.

REAL WORLD USAGE & TRADEOFF

Usage: Client attaches JWT token header to auth API requests.

Trade-off: Stateless API speed (JWT) vs immediate token revocation capacities (Stateful Sessions).

COMMON MISTAKE

Storing sensitive information like user passwords inside the public JWT payload.

INTERVIEWER TRAP

How do you handle JWT token revocation?

TOP FOLLOW-UP

Explain the difference between JWT access tokens and refresh tokens.

MEMORY TRICK

A JWT is a prepaid movie ticket: staff check the signature stamp without looking up database logs.

OAuth

DEFINITION & WHY IT EXISTS

Definition: An open standard for authorization, allowing users to share private resources without sharing passwords.

Why It Exists: Enables 'Login with Google/GitHub' integrations securely.

REAL WORLD USAGE & TRADEOFF

Usage: Allowing GrindOS dashboards to load a user's GitHub repository lists.

Trade-off: Developing custom identity servers vs relying on external OAuth providers.

COMMON MISTAKE

Failing to validate the state parameter, exposing systems to CSRF login attacks.

INTERVIEWER TRAP

Explain the OAuth 2.0 Authorization Code flow.

TOP FOLLOW-UP

What is OpenID Connect (OIDC)?

MEMORY TRICK

OAuth is valet keys: opens the driver's door and starts the car, but cannot open the glove box.

Session Auth

DEFINITION & WHY IT EXISTS

Definition: A stateful authentication method where session records are stored on the server.

Why It Exists: Enables immediate session revocation and tight control over active users.

REAL WORLD USAGE & TRADEOFF

Usage: Storing active session keys in Redis, verifying IDs on every HTTP request.

Trade-off: Immediate session control (Stateful Sessions) vs stateless scale capacities (JWT).

COMMON MISTAKE

Failing to set HttpOnly flags on session cookies, exposing them to XSS script theft.

INTERVIEWER TRAP

How does session hijacking occur and how do you prevent it?

TOP FOLLOW-UP

What are secure HttpOnly cookies?

MEMORY TRICK

Session auth is checking the guest list on every entry to a private party.

RBAC

DEFINITION & WHY IT EXISTS

Definition: Role-Based Access Control: restricting system access to authorized users based on roles.

Why It Exists: Simplifies permission mapping as corporate organizations scale.

REAL WORLD USAGE & TRADEOFF

Usage: Assigning roles: `admin`, `recruiter`, `student` to manage dashboard API scopes.

Trade-off: Simple role management setups (RBAC) vs complex context rule configurations (ABAC).

COMMON MISTAKE

Creating too many specialized roles (role explosion), making permission auditing difficult.

INTERVIEWER TRAP

How do you implement hierarchical roles in RBAC databases?

TOP FOLLOW-UP

What is the difference between roles and permissions?

MEMORY TRICK

RBAC is keys based on job titles: all managers get keys to the manager office.

ABAC

DEFINITION & WHY IT EXISTS

Definition: Attribute-Based Access Control: evaluating access rules dynamically using attributes.

Why It Exists: Enables fine-grained security policies based on context (time, location, department).

REAL WORLD USAGE & TRADEOFF

Usage: Allowing file access only if: ``user.department == document.department` AND `time.hour < 17`.`

Trade-off: Strict context security (ABAC) vs system performance and configuration simplicity (RBAC).

COMMON MISTAKE

Implementing ABAC when simple role checks are sufficient, slowing down API routing.

INTERVIEWER TRAP

How does XACML define ABAC policies?

TOP FOLLOW-UP

What attributes compile environment contexts in ABAC?

MEMORY TRICK

ABAC is checking ID, age, local time, and ticket status before entry to an R-rated movie.

Zero Trust

DEFINITION & WHY IT EXISTS

Definition: A security framework requiring continuous verification of every user and device, both inside and outside the network.

Why It Exists: Prevents lateral movement attacks if perimeter security fails.

REAL WORLD USAGE & TRADEOFF

Usage: Enforcing mTLS encryption and session validation for all internal microservices.

Trade-off: Open internal network speeds (perimeter security) vs absolute request verification security (Zero Trust).

COMMON MISTAKE

Assuming internal Kubernetes cluster traffic is safe by default, leaving API ports open.

INTERVIEWER TRAP

What is lateral movement in network security breaches?

TOP FOLLOW-UP

How do you implement micro-segmentation in Kubernetes?

MEMORY TRICK

Zero Trust is checking passenger tickets and ID at every single station during a train ride.

Logging

DEFINITION & WHY IT EXISTS

Definition: Recording application events, execution paths, and error stack traces to disk or storage.

Why It Exists: Provides historical records to diagnose production crashes and bugs.

REAL WORLD USAGE & TRADEOFF

Usage: Writing structured JSON logs: `{'level': 'ERROR', 'msg': 'DB connection failed'}`.

Trade-off: Detailed debug logging (high storage bills) vs minimal log tracking (harder to debug).

COMMON MISTAKE

Writing sensitive user data (passwords, credit card numbers) into logs, violating compliance.

INTERVIEWER TRAP

Why are structured logs (JSON) preferred over plaintext strings?

TOP FOLLOW-UP

What are log aggregation pipelines like ELK or EFK?

MEMORY TRICK

Logging is keeping a detailed diary of everything you did during the work day.

Monitoring

DEFINITION & WHY IT EXISTS

Definition: The real-time observation of system metrics, resource usage, and application health indicators.

Why It Exists: Ensures systems are healthy, alerting teams before outages occur.

REAL WORLD USAGE & TRADEOFF

Usage: Monitoring API error rates and CPU limits across microservice clusters.

Trade-off: Thorough monitoring systems (higher costs) vs minimal health tracking (reactive debug paths).

COMMON MISTAKE

Setting tight alert rules, causing alert fatigue and leading teams to ignore notifications.

INTERVIEWER TRAP

What are the four golden signals of monitoring?

TOP FOLLOW-UP

What is black-box vs white-box monitoring?

MEMORY TRICK

Monitoring is the dashboard gauges on a car: watch fuel levels and engine temp.

Metrics

📖 DEFINITION & WHY IT EXISTS

Definition: Numeric data measurements of system resource usage gathered over time.

Why It Exists: Provides quantitative data to identify usage trends and plan hardware expansions.

🌐 REAL WORLD USAGE & TRADEOFF

Usage: Gathering database memory allocations and CPU utilization trends.

Trade-off: High-frequency metrics gathering (detailed trends) vs low-frequency gathering (reduced storage costs).

⚠️ COMMON MISTAKE

Failing to index metric tags, slowing down dashboard load query runs.

🔴 INTERVIEWER TRAP

Explain counter, gauge, and histogram metric types.

💡 TOP FOLLOW-UP

What are time-series databases (TSDB)?

💡 MEMORY TRICK

Metrics are your smart watch tracking your heart rate throughout the day.

Alerting

DEFINITION & WHY IT EXISTS

Definition: The system process of sending notifications to teams when metrics cross specified safety limits.

Why It Exists: Notifies engineering teams of system issues before users experience downtime.

REAL WORLD USAGE & TRADEOFF

Usage: Alerting Slack channels if API endpoint return times exceed 2 seconds for 5 minutes.

Trade-off: Immediate pager alerts (resolves issues fast) vs quiet notifications (reduces alert fatigue).

COMMON MISTAKE

Sending non-critical warning alerts to on-call phone pages at midnight, burning out engineers.

INTERVIEWER TRAP

How do you design a reliable on-call escalation rotation path?

TOP FOLLOW-UP

What causes alert fatigue and how do you resolve it?

MEMORY TRICK

Alerting is the smoke detector in your kitchen: rings loudly to warn of fire.

Tracing

DEFINITION & WHY IT EXISTS

Definition: Tracking the execution path of single transactions as they traverse distributed services.

Why It Exists: Identifies microservice dependency delays and call bottlenecks.

REAL WORLD USAGE & TRADEOFF

Usage: Using OpenTelemetry to trace user checkout request paths.

Trade-off: 100% trace sampling (detailed logs, high cost) vs 5% request trace sampling (budget friendly).

COMMON MISTAKE

Sampling 100% of high-volume trace calls, causing log storage costs to skyrocket.

INTERVIEWER TRAP

How does distributed tracing help debug cascading timeouts?

TOP FOLLOW-UP

What are tracing spans?

MEMORY TRICK

Tracing is tracking a package through every transit hub using a barcode scanner.

Prometheus

DEFINITION & WHY IT EXISTS

Definition: A time-series monitoring system that pulls metrics from configured targets at regular intervals.

Why It Exists: Provides an open-source metric gathering system with query logic (PromQL).

REAL WORLD USAGE & TRADEOFF

Usage: Configuring Prometheus to scrape metrics from FastAPI app instances every 15 seconds.

Trade-off: Pull-based scraping (simple setup) vs push-based metrics pipelines (better for ephemeral tasks).

COMMON MISTAKE

Relying on Prometheus for long-term data archiving without configuring external stores like Thanos.

INTERVIEWER TRAP

Explain pull vs push metrics collection paradigms.

TOP FOLLOW-UP

What is PromQL?

MEMORY TRICK

Prometheus is a teacher walking around the class, reading grades from student desks.

Grafana

DEFINITION & WHY IT EXISTS

Definition: An open-source visualization and analytics software system to query and visualize metrics.

Why It Exists: Renders metrics databases as readable dashboards.

REAL WORLD USAGE & TRADEOFF

Usage: Connecting Grafana to Prometheus, displaying live web traffic graphs on office screens.

Trade-off: Custom dashboard creation vs predefined monitoring setups.

COMMON MISTAKE

Creating overly complex dashboards with too many charts, slowing down page loads.

INTERVIEWER TRAP

How do you configure dynamic variables inside Grafana dashboards?

TOP FOLLOW-UP

What data sources does Grafana support natively?

MEMORY TRICK

Grafana is the TV screen showing live charts and graphs in the flight control tower.

URL Shortener

DEFINITION & WHY IT EXISTS

Definition: A system design case study for a service that creates short aliases for long URLs (like tinyurl.com).

Why It Exists: Demonstrates capacity estimation, database selections, and unique key generation logic.

REAL WORLD USAGE & TRADEOFF

Usage: Converting 'https://grindos.pranavx.in/page' into a 7-character string 'tinyurl.com/a7B2x'.

Trade-off: Base62 conversion on write vs pre-generating unique keys in a registry database.

COMMON MISTAKE

Pre-generating unique keys using databases without unique constraint checks, risking duplicate key collision bugs.

INTERVIEWER TRAP

Why choose Base62 encoding over Base64 encoding for URL paths?

TOP FOLLOW-UP

How do you handle redirect routing using HTTP 301 vs 302 statuses?

MEMORY TRICK

A URL shortener is a coat check ticket: exchange your big coat for a small plastic tag.

WhatsApp

DEFINITION & WHY IT EXISTS

Definition: Designing a real-time messaging system supporting chat, delivery indicators, and group conversations.

Why It Exists: Demonstrates real-time WebSocket scaling, connection mapping, and message queuing.

REAL WORLD USAGE & TRADEOFF

Usage: Orchestrating messages across millions of mobile client applications globally.

Trade-off: Stateful persistent WebSockets vs stateless HTTP requests polling.

COMMON MISTAKE

Storing chat history in central SQL tables without partitioning, slowing queries.

INTERVIEWER TRAP

How do you scale WebSocket servers to support millions of active connections?

TOP FOLLOW-UP

How do you handle chat indicators (delivered, read) asynchronously?

MEMORY TRICK

WhatsApp is a post office sorting room: route messages directly to active carrier routes.

Instagram Feed

DEFINITION & WHY IT EXISTS

Definition: Designing a social media feed system supporting photo uploads, user follows, and dynamic home page feeds.

Why It Exists: Demonstrates hybrid feed generation models (push vs pull) for active user groups.

REAL WORLD USAGE & TRADEOFF

Usage: Generating home page content feeds for millions of active social media users.

Trade-off: Pre-computed push models (fast reads, slow writes) vs pull query systems (slow reads, fast writes).

COMMON MISTAKE

Running dynamic relational join queries on feed loads, crashing databases under active use.

INTERVIEWER TRAP

How do you handle celebrity posts (celebrity problem) in feed generation?

TOP FOLLOW-UP

Explain feed cache eviction strategies.

MEMORY TRICK

Instagram feed is printing newspaper copies: print early so users read instantly.

YouTube

DEFINITION & WHY IT EXISTS

Definition: Designing a video streaming platform supporting uploads, transcoding, and video recommendations.

Why It Exists: Demonstrates blob storage management, transcode queues, and CDN delivery networks.

REAL WORLD USAGE & TRADEOFF

Usage: Delivering video content to global audiences across varying network configurations.

Trade-off: Higher transcode server costs vs low client playback buffering latency.

COMMON MISTAKE

Exposing primary storage buckets to direct client downloads, bypassing CDN cache nodes.

INTERVIEWER TRAP

Why are video files split into chunks during transcoding?

TOP FOLLOW-UP

How do recommendations engines load dynamic suggestion feeds?

MEMORY TRICK

YouTube is a translator: translates raw text into multiple languages for regional readers.

Uber

DEFINITION & WHY IT EXISTS

Definition: Designing a ride-sharing dispatch system mapping dynamic coordinates and matching riders with drivers.

Why It Exists: Demonstrates geospatial database indexing, coordinate streaming, and match algorithms.

REAL WORLD USAGE & TRADEOFF

Usage: Matching active passenger ride requests with nearby drivers in under 2 seconds.

Trade-off: Real-time dispatch accuracy vs memory database compute costs.

COMMON MISTAKE

Using flat SQL queries with mathematical range calculations to locate drivers, slowing database runs.

INTERVIEWER TRAP

Explain spatial indexing models: Quadtree vs Google S2 vs Uber H3.

TOP FOLLOW-UP

How do you handle coordinate update bottlenecks?

MEMORY TRICK

Uber is a dispatcher with a map: group coordinates by grid sections to find nearest drivers.

Zomato

DEFINITION & WHY IT EXISTS

Definition: Designing a food delivery platform matching restaurants, drivers, and delivery routes.

Why It Exists: Demonstrates search path filtering, route calculations, and order transactions.

REAL WORLD USAGE & TRADEOFF

Usage: Routing order requests to restaurants and delivery drivers in real-time.

Trade-off: Accurate dynamic route updates vs server calculate latencies.

COMMON MISTAKE

Directly querying primary databases for menu catalogs, slowing check transactions.

INTERVIEWER TRAP

How do you scale catalog cache databases under high check traffic?

TOP FOLLOW-UP

How does order status tracking sync across clients?

MEMORY TRICK

Zomato is a dispatcher coordinating food pickup and delivery routes.

Google Drive

DEFINITION & WHY IT EXISTS

Definition: Designing a file cloud storage and synchronization system supporting file uploads and edit syncs.

Why It Exists: Demonstrates block storage slice logic, sync engines, and metadata databases.

REAL WORLD USAGE & TRADEOFF

Usage: Storing and synchronizing files across multiple user devices.

Trade-off: Simple file replacement uploads vs block-level differential updates.

COMMON MISTAKE

Uploading full documents on minor edits, draining user data limits.

INTERVIEWER TRAP

Why divide files into block-level chunks for storage?

TOP FOLLOW-UP

How do block indexes optimize file synchronizations?

MEMORY TRICK

Google Drive is building modular homes: only ship modified parts to the site.

Notification System

DEFINITION & WHY IT EXISTS

Definition: Designing a multi-channel notification platform delivering SMS, email, and push alerts.

Why It Exists: Demonstrates message queuing, provider integration, and rate limiting.

REAL WORLD USAGE & TRADEOFF

Usage: Dispatching verification codes, newsletters, and order status alerts to users.

Trade-off: Immediate synchronous API calls vs asynchronous background queues.

COMMON MISTAKE

Calling notification APIs inline, causing main application thread delays.

INTERVIEWER TRAP

How do you prevent duplicate alerts on message retry loops?

TOP FOLLOW-UP

Explain notification provider fallback routines.

MEMORY TRICK

A notification system is a call center: queue requests and delegate calls to workers.

Chat System

DEFINITION & WHY IT EXISTS

Definition: Designing a real-time messaging system supporting private chats and online status tracking.

Why It Exists: Demonstrates WebSocket scaling, message queues, and session caching.

REAL WORLD USAGE & TRADEOFF

Usage: Broadcasting messages across active user chat window channels in real-time.

Trade-off: Stateless polling vs stateful persistent WebSockets.

COMMON MISTAKE

Storing transient chat message details directly in relational DB tables, slowing system writes.

INTERVIEWER TRAP

How do you scale user online status databases under high traffic?

TOP FOLLOW-UP

How do you handle message delivery confirmations?

MEMORY TRICK

A chat system is a telephone exchange: route lines directly between callers.

Rate Limiter

DEFINITION & WHY IT EXISTS

Definition: Designing a system to limit HTTP request rates based on IP addresses or API keys.

Why It Exists: Demonstrates low-latency memory stores, token buckets, and sliding window designs.

REAL WORLD USAGE & TRADEOFF

Usage: Protecting payment endpoints by restricting transactions to 5 per minute per card.

Trade-off: Token bucket algorithms (burst support) vs leaky bucket algorithms (smooth traffic).

COMMON MISTAKE

Running rate limiter key reads on primary SQL databases, causing performance drops.

INTERVIEWER TRAP

Explain sliding window counter rate limiting algorithms.

TOP FOLLOW-UP

How do you coordinate rate limits across server clusters?

MEMORY TRICK

A rate limiter is a security bouncer: only allow guests inside at a safe pace.

Requirement Gathering

DEFINITION & WHY IT EXISTS

Definition: The step in system design interviews to clarify ambiguities and define functional and non-functional requirements.

Why It Exists: Prevents building the wrong system and outlines design bounds.

REAL WORLD USAGE & TRADEOFF

Usage: Asking: 'How many daily active users?' and 'Should reads be real-time?' before planning.

Trade-off: Thorough requirements discussions vs early design phase starts.

COMMON MISTAKE

Assuming requirements without asking. This shows poor developer communication.

INTERVIEWER TRAP

What questions define non-functional SLA requirements?

TOP FOLLOW-UP

How do you prioritize requirements during time limits?

MEMORY TRICK

Requirement gathering is getting blue-prints from a client before building.

Capacity Estimation

DEFINITION & WHY IT EXISTS

Definition: Calculating system scale bounds (storage, memory, network bandwidth) during interview planning.

Why It Exists: Identifies system bottlenecks and guides database and hardware sizing choices.

REAL WORLD USAGE & TRADEOFF

Usage: Calculating storage needs: 10M active users * 100 bytes = 1GB storage per day.

Trade-off: Accurate precision calculations vs quick, realistic range approximations.

COMMON MISTAKE

Failing to convert raw metrics to standard sizes (e.g., converting bits to bytes).

INTERVIEWER TRAP

How do you calculate network bandwidth limits?

TOP FOLLOW-UP

Explain standard memory size bounds for caching.

MEMORY TRICK

Capacity estimation is budget planning: estimate costs before buying furniture.

API Design

DEFINITION & WHY IT EXISTS

Definition: Defining the REST/gRPC endpoints, request structures, and HTTP response codes for a system.

Why It Exists: Establishes clear communication contracts between clients and backend servers.

REAL WORLD USAGE & TRADEOFF

Usage: Designing route: `POST /v1/trips` with JSON request body parameters.

Trade-off: REST API simplicity vs gRPC performance and payload size advantages.

COMMON MISTAKE

Using generic HTTP 200 codes for error events, complicating client-side error handling.

INTERVIEWER TRAP

What HTTP status codes represent user authentication issues?

TOP FOLLOW-UP

Explain idempotency keys inside API header definitions.

MEMORY TRICK

API design is drawing up a contract: define terms before signing.

Database Design

DEFINITION & WHY IT EXISTS

Definition: Selecting databases and designing table schemas, indices, and data paths.

Why It Exists: Ensures transaction safety, data integrity, and query performance.

REAL WORLD USAGE & TRADEOFF

Usage: Mapping relational tables, setting indexes, and outlining sharding strategies.

Trade-off: Relational SQL normalizations vs NoSQL document read performance.

COMMON MISTAKE

Neglecting index placement on foreign key columns, causing table join slowdowns.

INTERVIEWER TRAP

When should you denormalize database schemas?

TOP FOLLOW-UP

How do composite indexes optimize query runs?

MEMORY TRICK

Database design is laying foundation blocks: build solid layers first.

Bottleneck Analysis

DEFINITION & WHY IT EXISTS

Definition: Identifying performance bottlenecks and scaling constraints in system layouts.

Why It Exists: Prevents system failures under high concurrent user loads.

REAL WORLD USAGE & TRADEOFF

Usage: Analyzing database connections pool metrics during traffic spikes.

Trade-off: System performance checks vs application feature delivery velocity.

COMMON MISTAKE

Assuming scaling memory limits resolves database write bottlenecks.

INTERVIEWER TRAP

How do you detect database query locks?

TOP FOLLOW-UP

What metrics indicate CPU bus limitations?

MEMORY TRICK

Bottleneck analysis is locating traffic jams on highway maps.

Scaling Strategy

📖 DEFINITION & WHY IT EXISTS

Definition: Designing the scalability roadmap to transition from single instances to distributed clusters.

Why It Exists: Enables systems to scale smoothly under increasing user demands.

🌐 REAL WORLD USAGE & TRADEOFF

Usage: Integrating load balancers, caching, and database read-replicas.

Trade-off: Proactive auto-scaling configurations vs on-demand resource provisioning costs.

⚠️ COMMON MISTAKE

Scaling server pools before optimizing database query run configurations.

🔴 INTERVIEWER TRAP

How does DNS routing optimize global system scaling?

💡 TOP FOLLOW-UP

What are scaling issues in microservice messaging queues?

💡 MEMORY TRICK

Scaling strategy is adding engines to trains: pull heavier loads safely.

Tradeoff Discussion

DEFINITION & WHY IT EXISTS

Definition: Analyzing the pros and cons of system choices during architectural evaluations.

Why It Exists: Demonstrates engineering judgment and reasoning over blind technology choices.

REAL WORLD USAGE & TRADEOFF

Usage: Comparing SQL ACID consistency benefits against NoSQL horizontal write performance.

Trade-off: Strict data security models vs API response latency profiles.

COMMON MISTAKE

Claiming a technology choice has 'zero drawbacks'. Every design choice has tradeoffs.

INTERVIEWER TRAP

What tradeoffs define database selection decisions?

TOP FOLLOW-UP

How does latency relate to system consistency choices?

MEMORY TRICK

Tradeoff discussion is buying a house: balance price, location, and space.

Common Mistakes

📖 DEFINITION & WHY IT EXISTS

Definition: Identifying and avoiding critical design anti-patterns during system planning.

Why It Exists: Prevents architectural flaws that cause production crashes or data loss.

🌐 REAL WORLD USAGE & TRADEOFF

Usage: Avoiding single points of failure (SPOF) in load balancer configurations.

Trade-off: Simple fast layouts vs highly resilient redundant designs.

⚠️ COMMON MISTAKE

Failing to set timeouts on external API connections, locking server threads.

🔴 INTERVIEWER TRAP

What system configurations cause split-brain issues?

💡 TOP FOLLOW-UP

Explain N+1 query patterns in ORM databases.

💡 MEMORY TRICK

Common mistakes is knowing where potholes are on roads to drive safely.

Interview Framework

DEFINITION & WHY IT EXISTS

Definition: The step-by-step structure to navigate system design interviews in under 45 minutes.

Why It Exists: Ensures all key design areas (requirements, APIs, databases, scaling) are covered systematically.

REAL WORLD USAGE & TRADEOFF

Usage: Allocating: 5m requirements → 10m API/schema → 15m design → 10m scaling.

Trade-off: Deep dive discussion details vs complete system overview delivery.

COMMON MISTAKE

Jumping straight into drawing databases before clarifying requirements.

INTERVIEWER TRAP

How do you recover if you go off-track during interviews?

TOP FOLLOW-UP

What questions confirm interviewer alignment?

MEMORY TRICK

Interview framework is a flight checklist: verify all points before taking off.

Top 100 Questions

DEFINITION & WHY IT EXISTS

Definition: A list of high-yield system design interview questions and model answers.

Why It Exists: Prepares candidates to answer common architectural design questions confidently.

REAL WORLD USAGE & TRADEOFF

Usage: Reviewing questions on load balancing, caching, databases, and microservices.

Trade-off: Rote memorization of answers vs understanding core system design principles.

COMMON MISTAKE

Giving generic answers without context. Always tailor your design to the problem's SLAs.

INTERVIEWER TRAP

What are the most common database scaling questions?

TOP FOLLOW-UP

How do you explain service mesh configurations clearly?

MEMORY TRICK

Preparation is studying past maps before embarking on a journey.

Personal Notes & System Designs

